

Performance Engineering of Software Systems

LECTURE 18 GPU ARCHITECTURE

Xuhao Chen

April 20, 2025



What is a GPU?

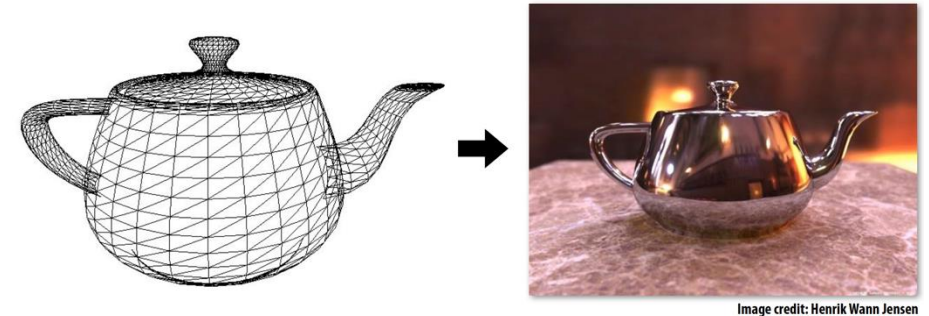
- Graphics Processing Units



ray tracing



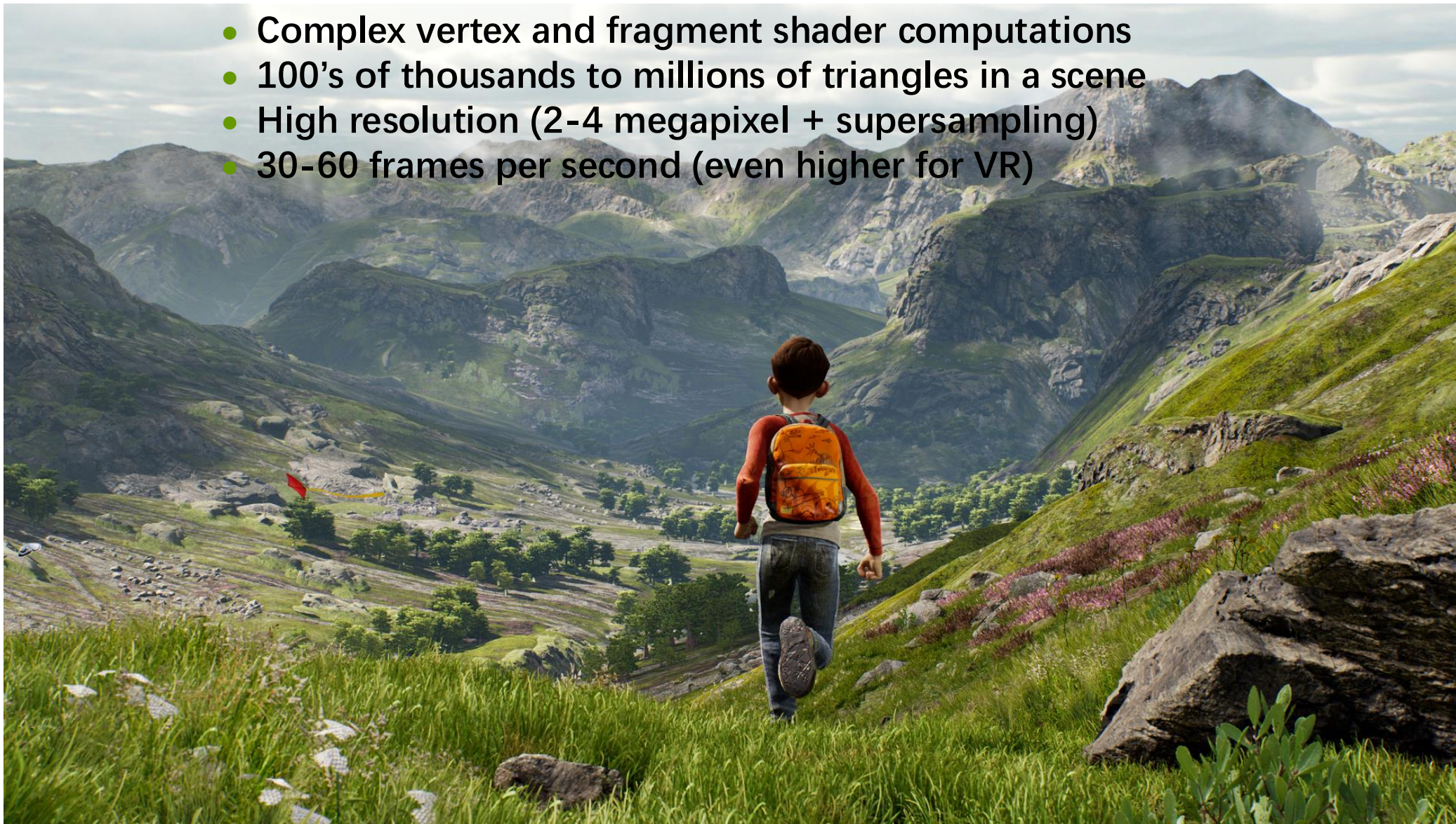
gaming



3D rendering

Goal: Highly Complex 3D Scenes in Realtime

- Complex vertex and fragment shader computations
- 100's of thousands to millions of triangles in a scene
- High resolution (2-4 megapixel + supersampling)
- 30-60 frames per second (even higher for VR)



A diffuse reflectance shader

```
sampler mySamp;  
Texture2D<float3> myTex;  
float3 lightDir;  
  
float4 diffuseShader(float3 norm, float2 uv)  
{  
    float3 kd;  
    kd = myTex.Sample(mySamp, uv);  
    kd *= clamp( dot(lightDir, norm), 0.0, 1.0);  
    return float4(kd, 1.0);  
}
```

- **How much compute is this?**

4 multiply-adds & 1 texture fetch

4K = 8 MPixels

x 5x Overdraw = 40 MPixels / frame

x 60hz = 2.4 GPixels/sec

~10 GFLOPS

~10 GB/sec

A real game is 10s to 100s of times more!

Outline

- GPU Architecture
 - ▣ Three major ideas that make GPU processing cores run fast
 - ▣ Closer look at real GPU designs
 - ▣ The GPU memory hierarchy: moving data to processors
- General Purpose GPU (GPGPU)
- GPU Programming Model
- Program a GPU with CUDA

GPU ARCHITECTURE: THE THREE BIG IDEAS

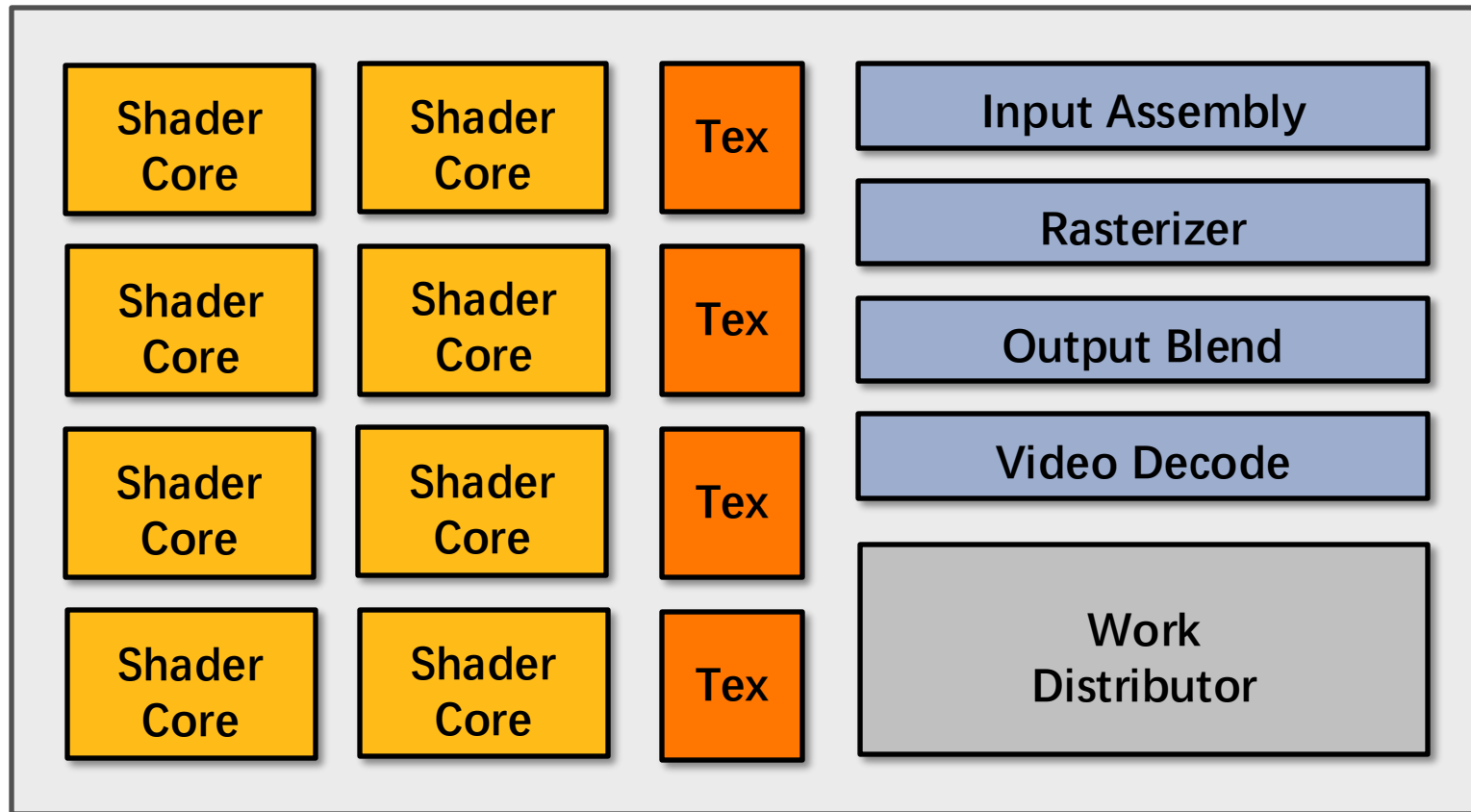


Throughput processing

- Three **key concepts** behind how modern GPUs run code
- Knowing these concepts will help you:
 - ▣ Understand space of GPU core (and throughput-oriented CPU core) designs
 - ▣ Understand how “GPU” cores do (and don’t!) differ from “CPU” cores
 - ▣ Optimize shaders/compute kernels
 - ▣ Establish intuition: what workloads might benefit from the design of these architectures?

What's in a GPU?

- A GPU is a heterogeneous chip multi-processor (highly tuned for graphics)



A diffuse reflectance shader

```
sampler mySamp;
Texture2D<float3> myTex;
float3 lightDir;

float4 diffuseShader(float3 norm, float2 uv)
{
    float3 kd;
    kd = myTex.Sample(mySamp, uv);
    kd *= clamp( dot(lightDir, norm), 0.0, 1.0);
    return float4(kd, 1.0);
}
```

- Shader programming model
 - ▣ Fragments are processed **independently**,
 - ▣ but there is no explicit parallel programming

Compile shader

```
sampler mySamp;  
Texture2D<float3> myTex;  
float3 lightDir;  
  
float4 diffuseShader(float3 norm, float2 uv)  
{  
    float3 kd;  
    kd = myTex.Sample(mySamp, uv);  
    kd *= clamp( dot(lightDir, norm), 0.0, 1.0);  
    return float4(kd, 1.0);  
}
```

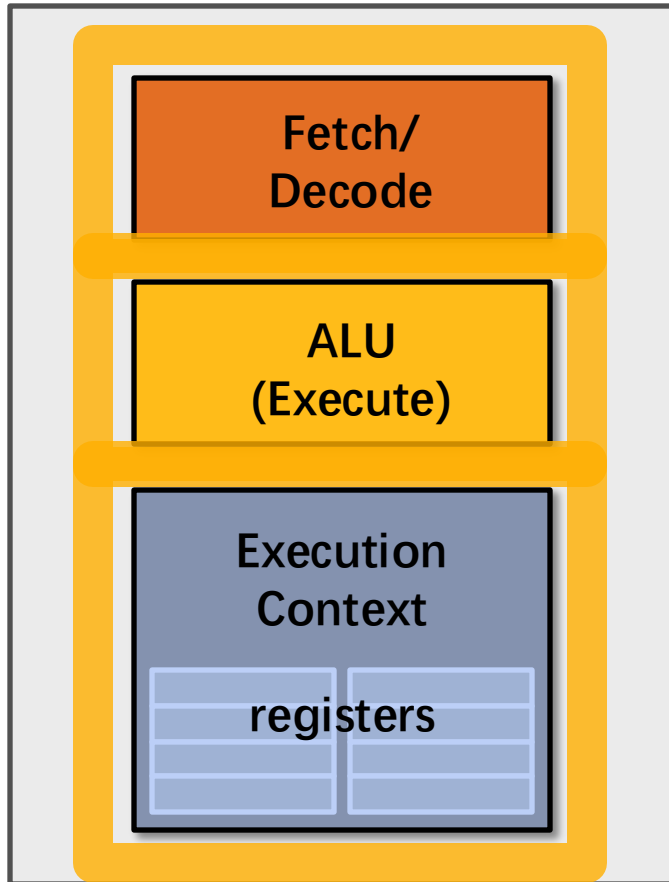
ed fragment input record 



```
<diffuseShader>:  
sample r0, v4, t0, s0  
mul   r3, v0, cb0[0]  
madd  r3, v1, cb0[1], r3  
madd  r3, v2, cb0[2], r3  
clmp  r3, r3, 1(0.0), 1(1.0)  
mul   o0, r0, r3  
mul   o1, r1, r3  
mul   o2, r2, r3  
mov   o3, 1(1.0)
```

1 unshaded fragment output record 

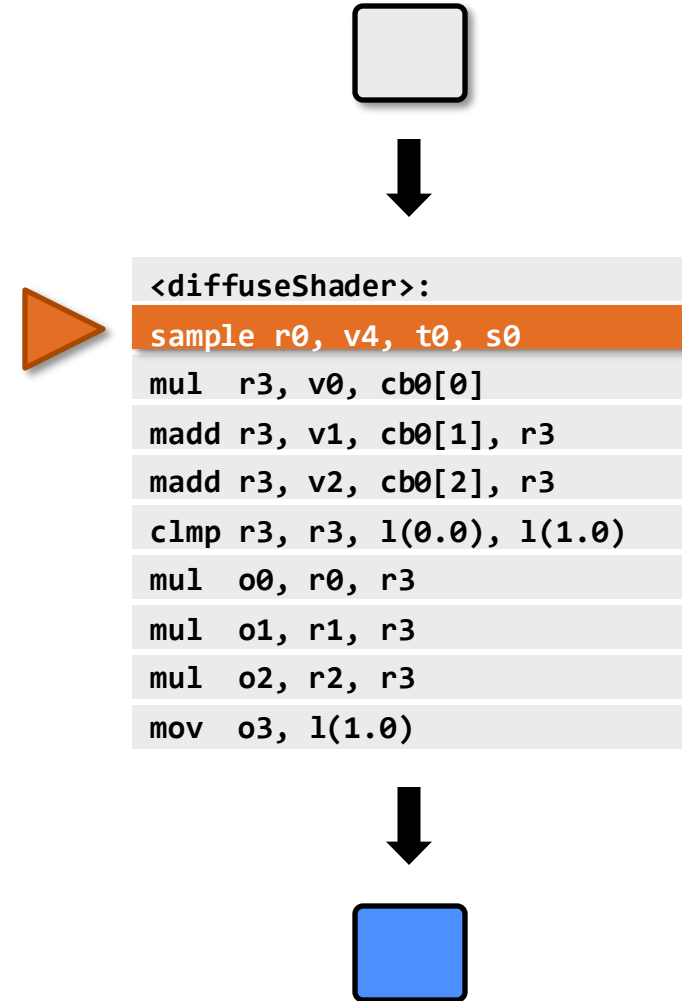
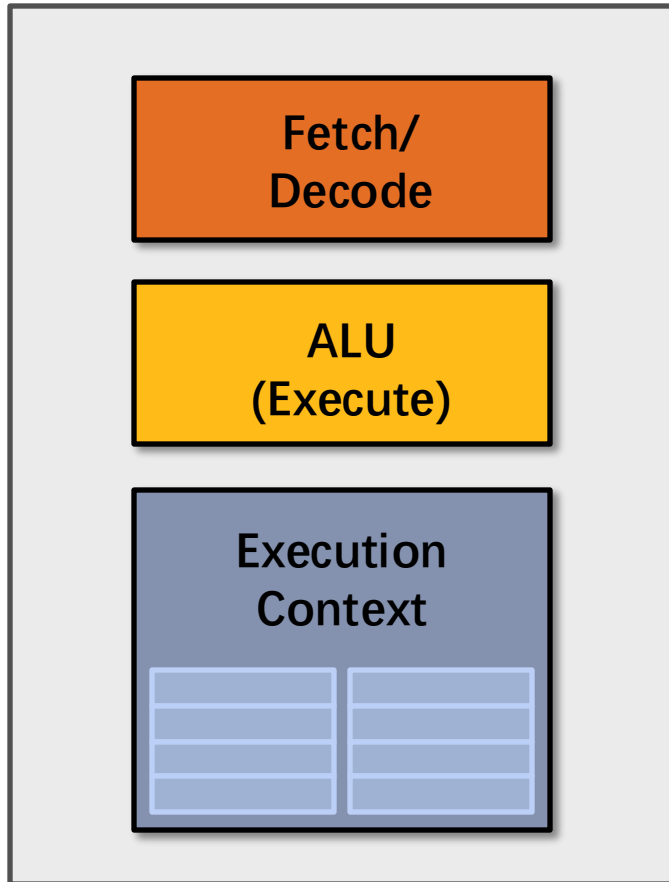
Execute shader



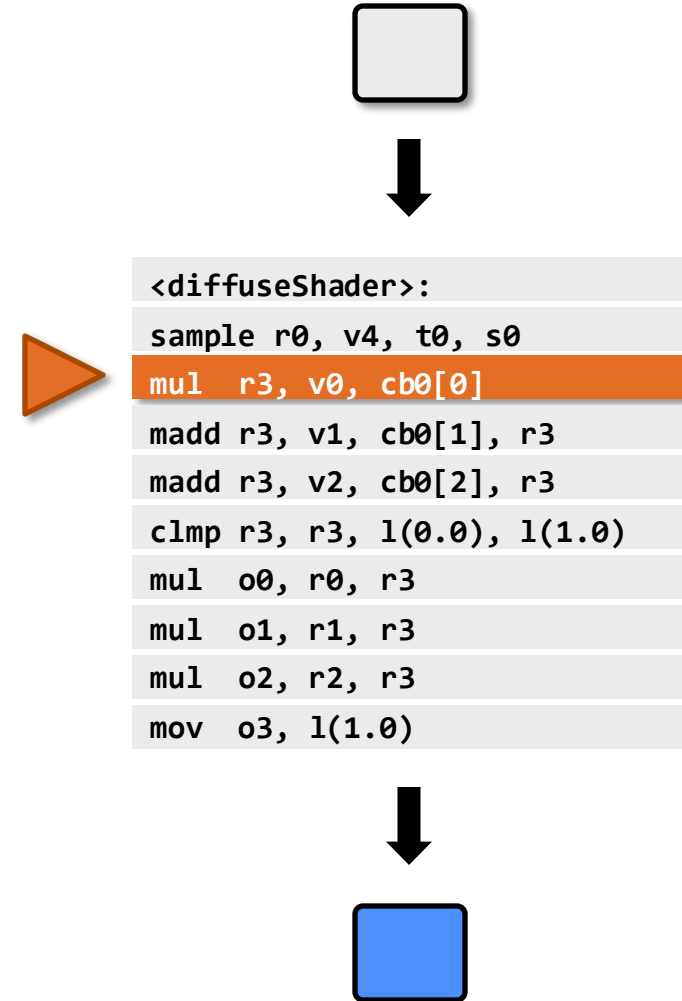
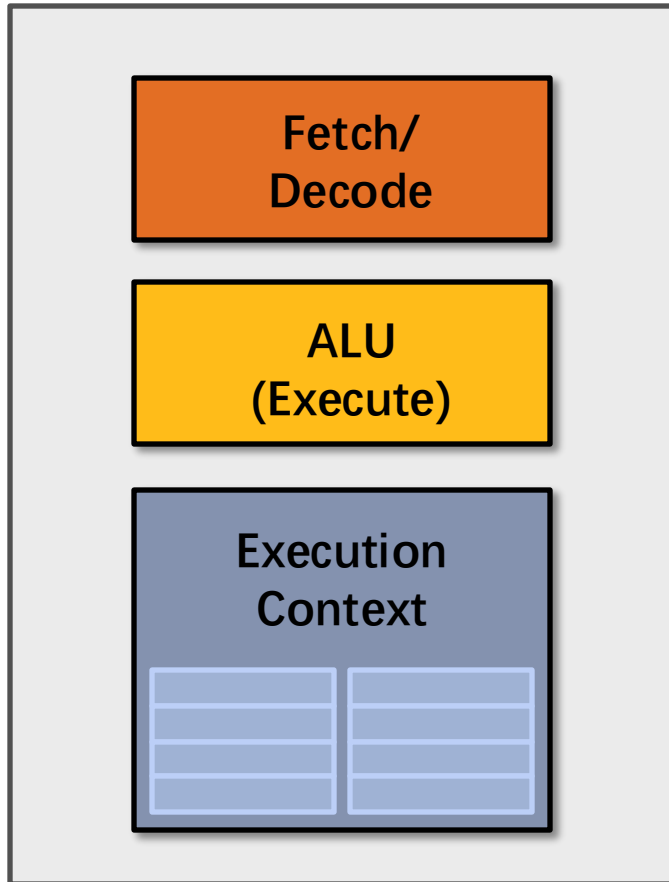
```
<diffuseShader>:  
sample r0, v4, t0, s0  
mul  r3, v0, cb0[0]  
madd r3, v1, cb0[1], r3  
madd r3, v2, cb0[2], r3  
clmp r3, r3, l(0.0), l(1.0)  
mul  o0, r0, r3  
mul  o1, r1, r3  
mul  o2, r2, r3  
mov  o3, l(1.0)
```



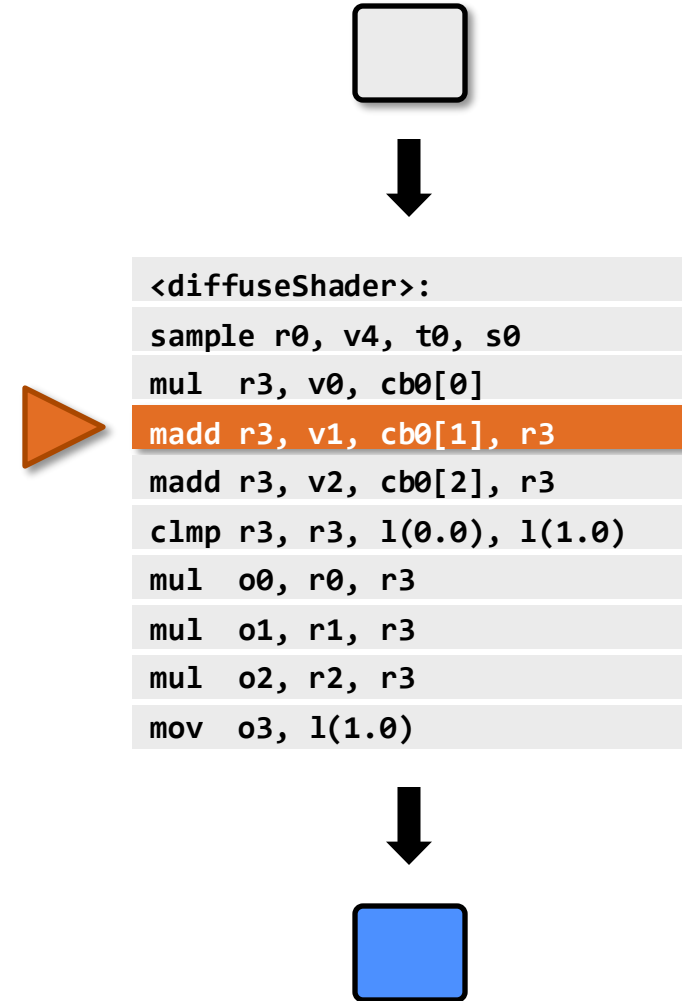
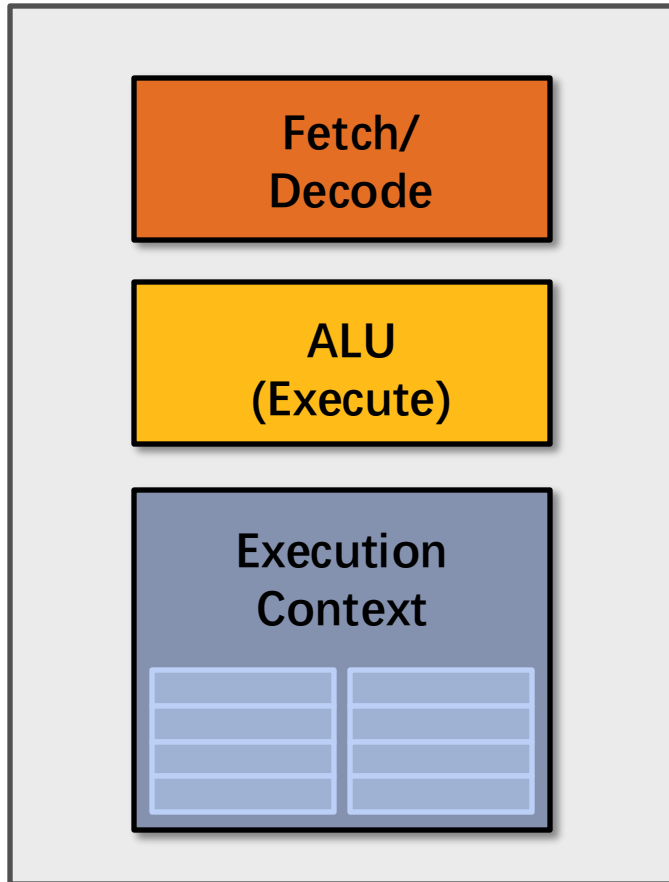
Execute shader



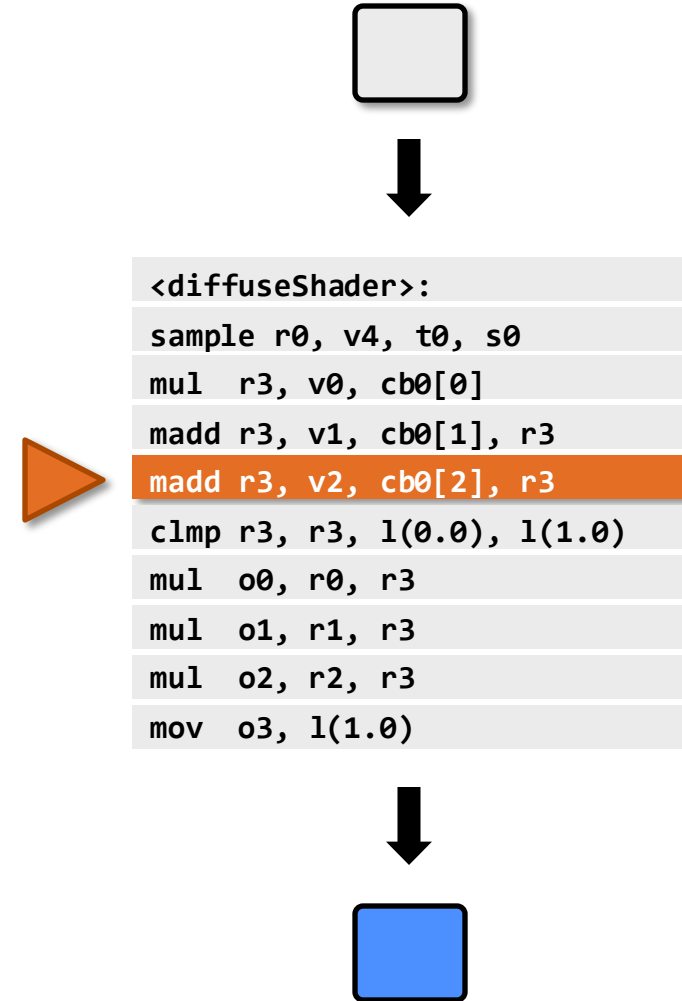
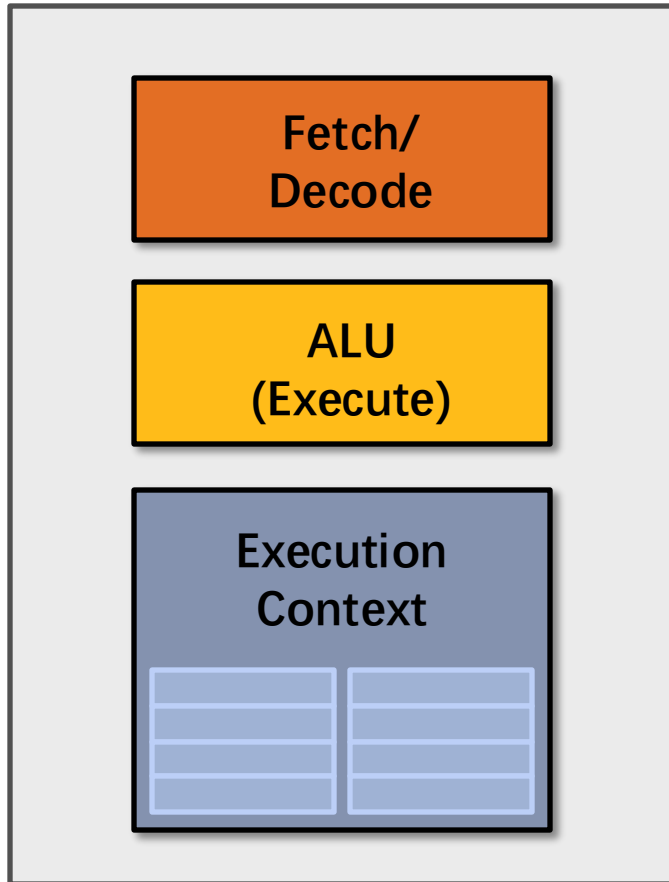
Execute shader



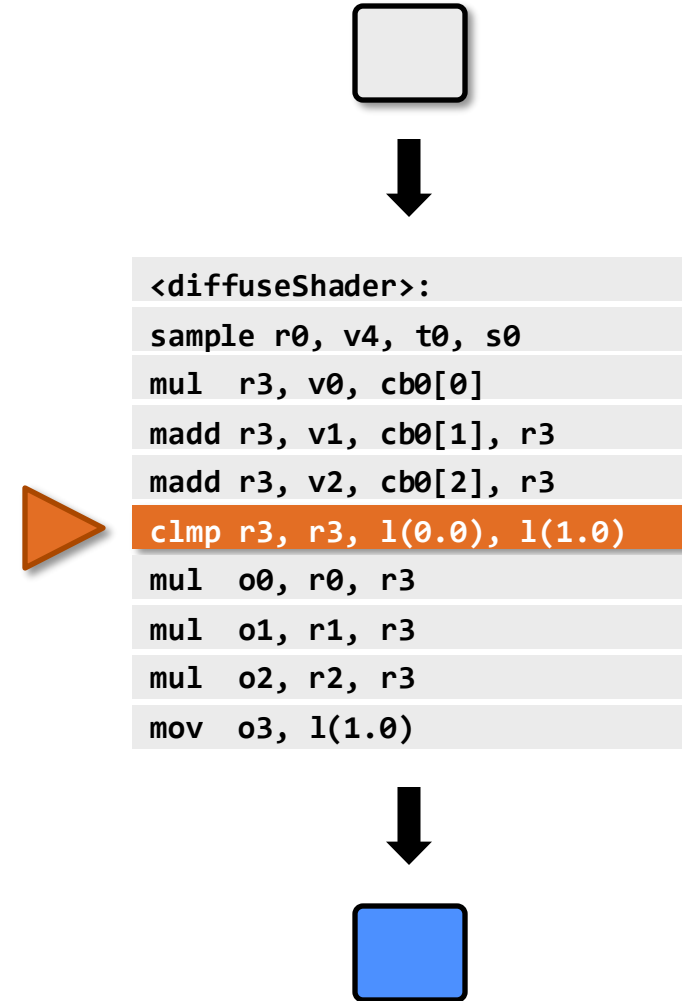
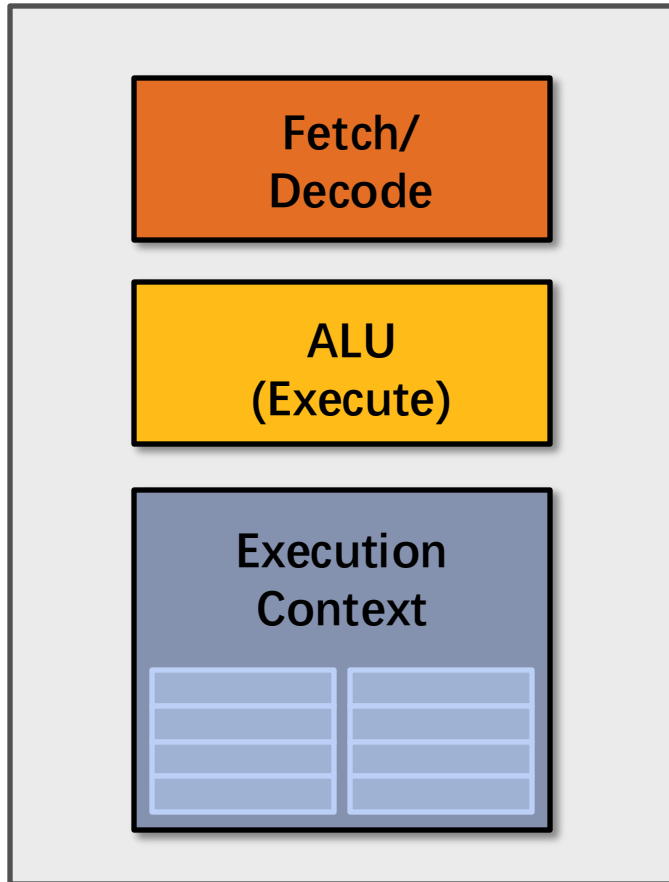
Execute shader



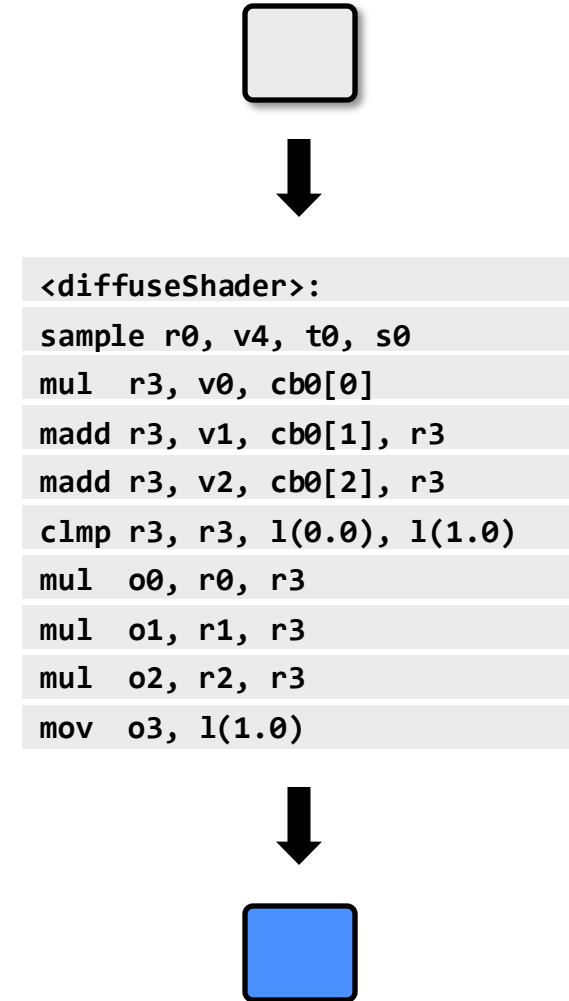
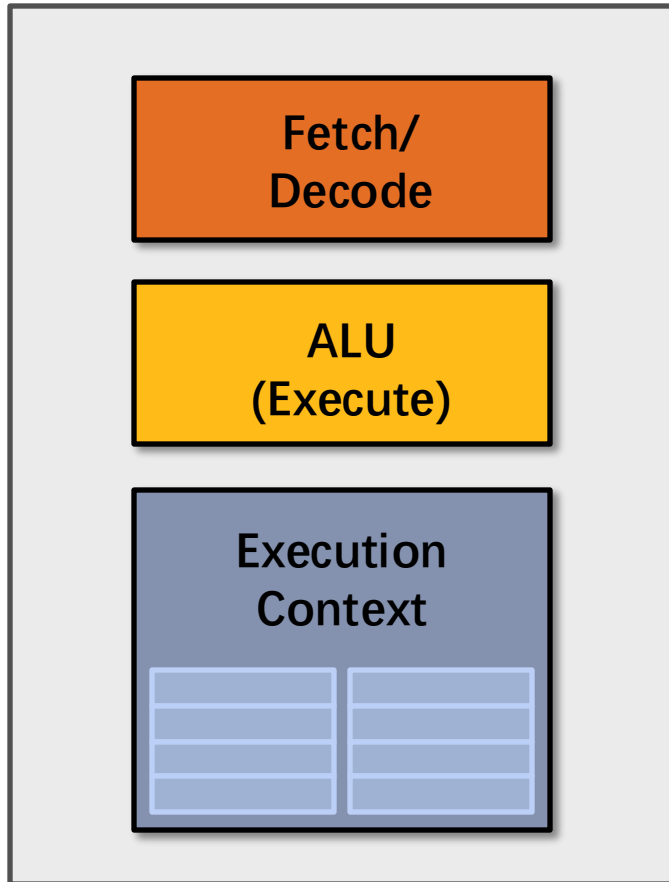
Execute shader



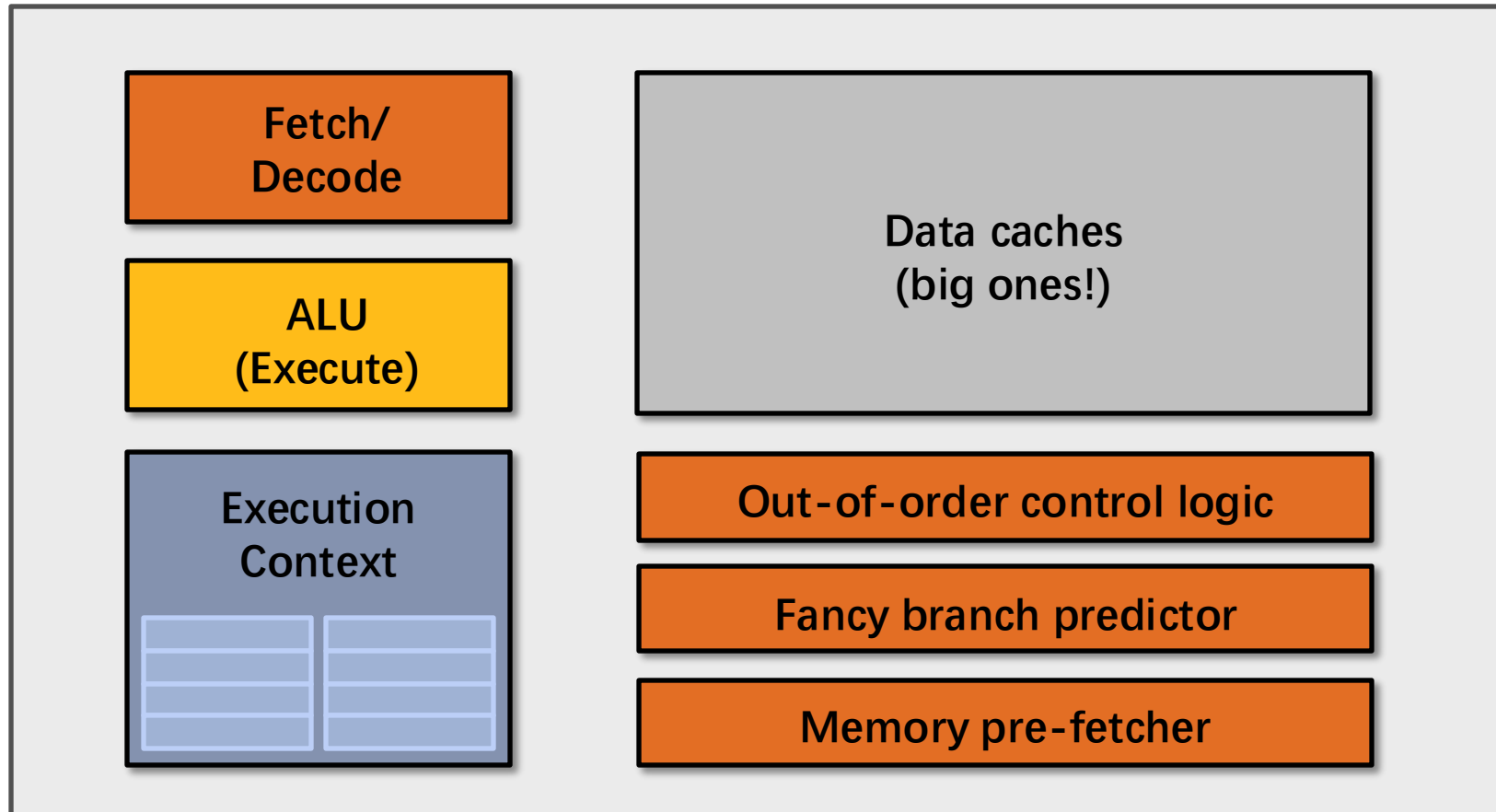
Execute shader



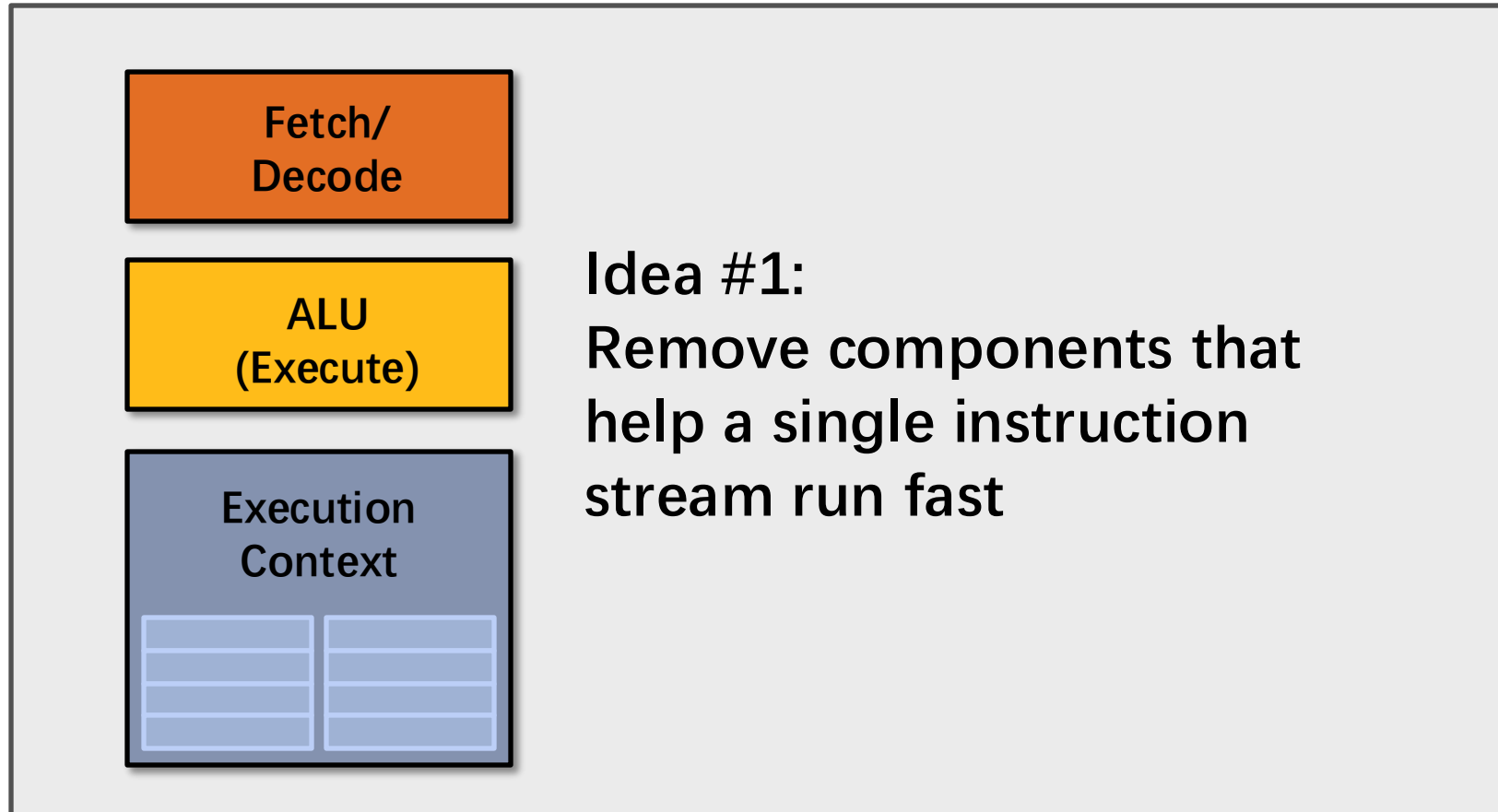
Execute shader



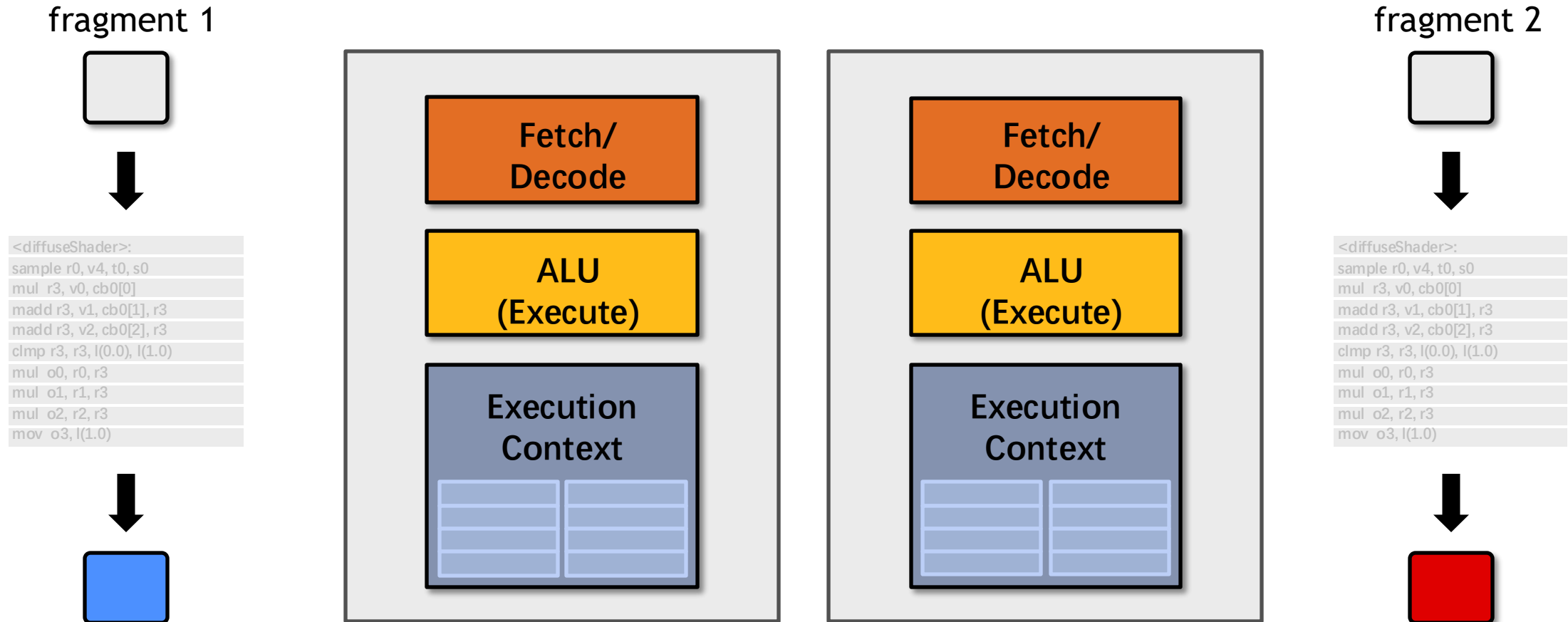
“CPU-style” cores



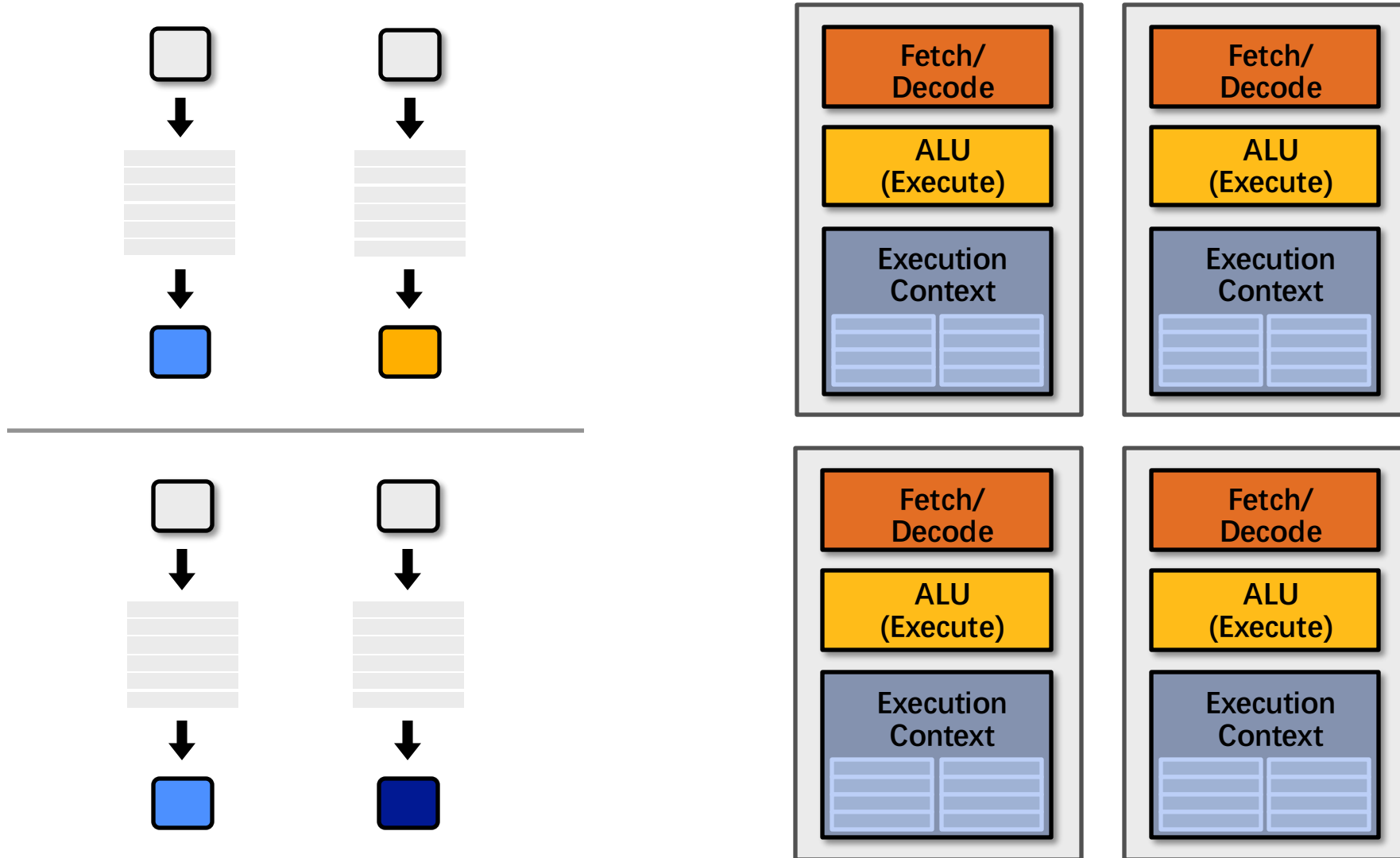
Slimming down



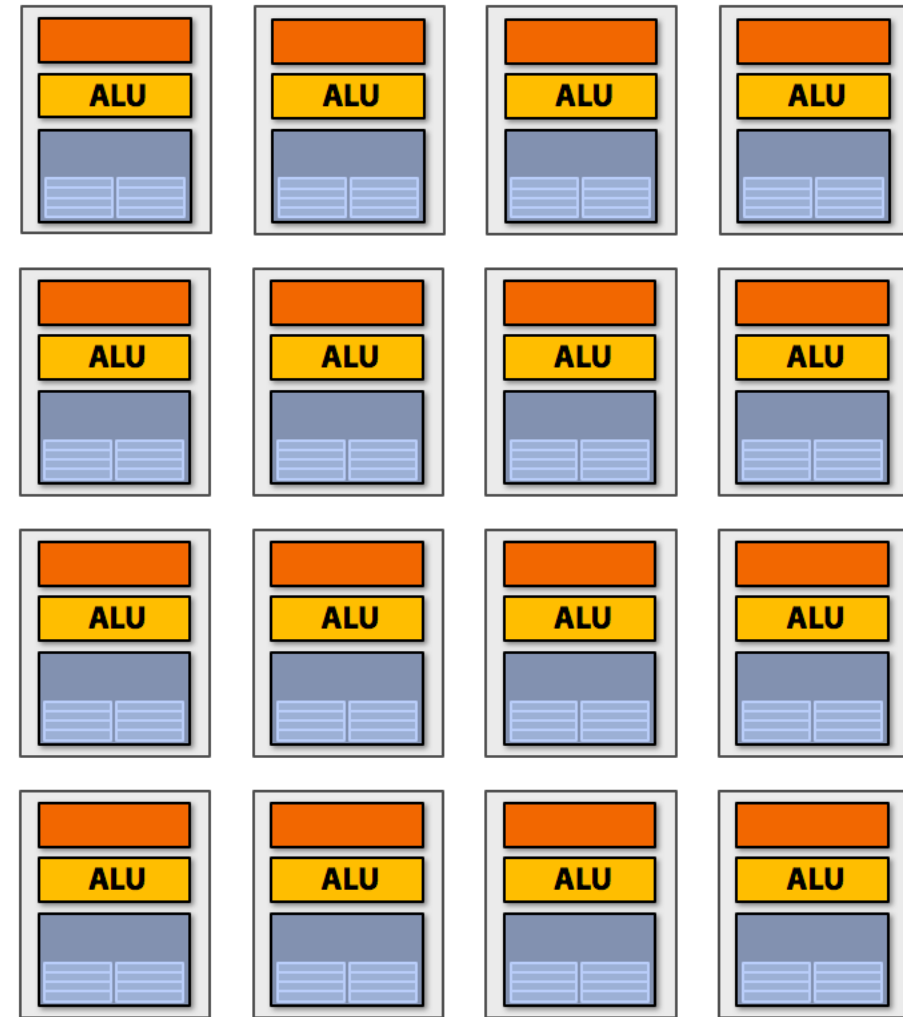
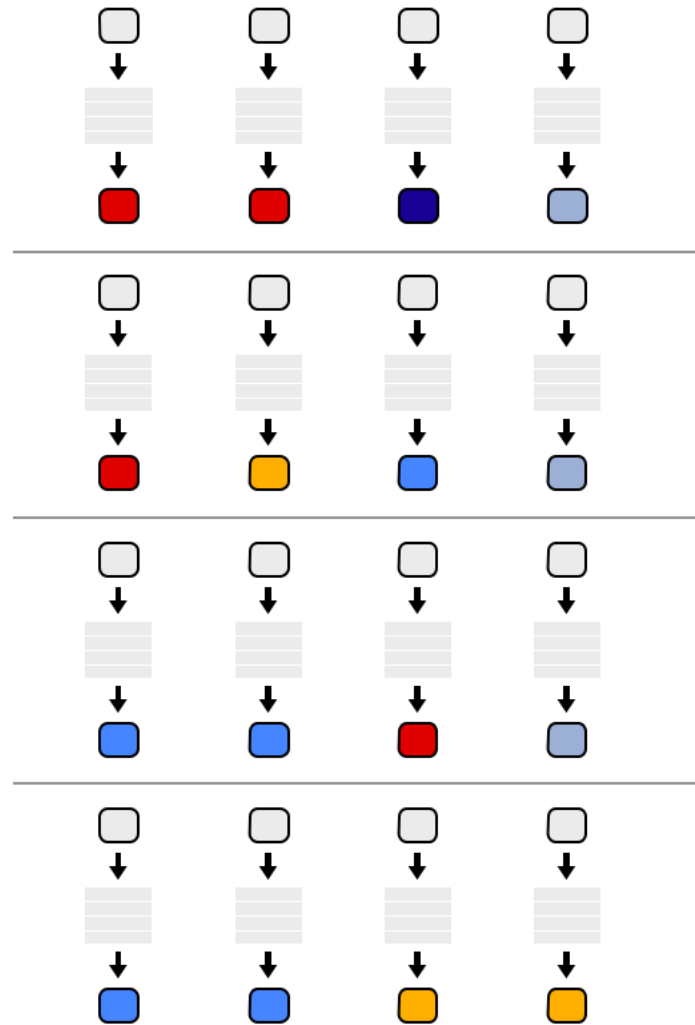
Two cores (two fragments in parallel)



Four cores (four fragments in parallel)

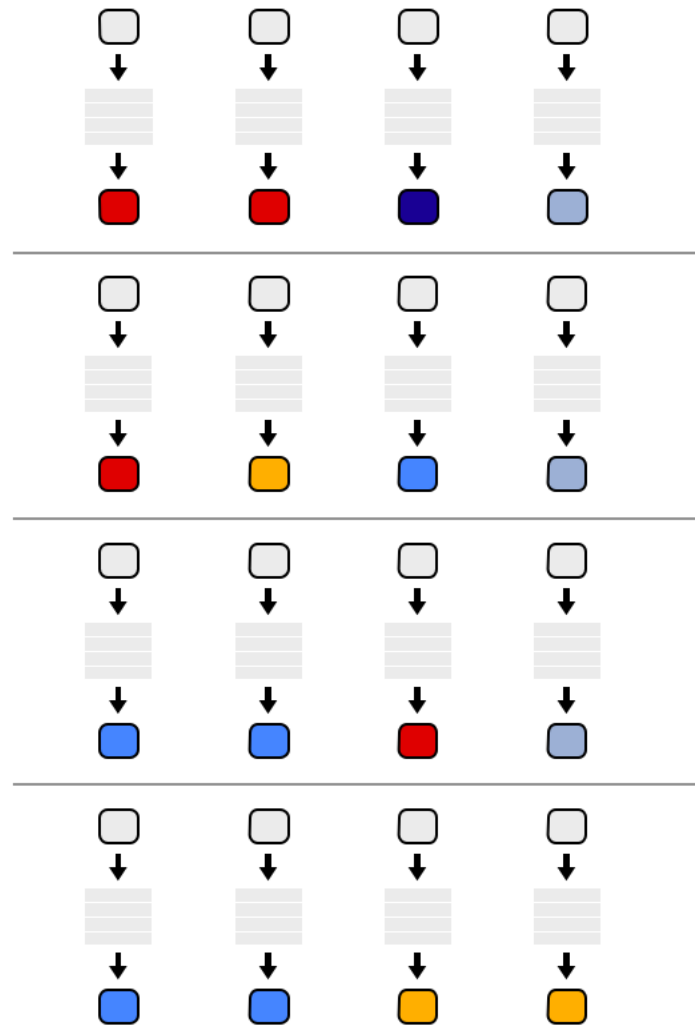


Sixteen cores (sixteen fragments in parallel)



16 cores = 16 simultaneous instruction streams

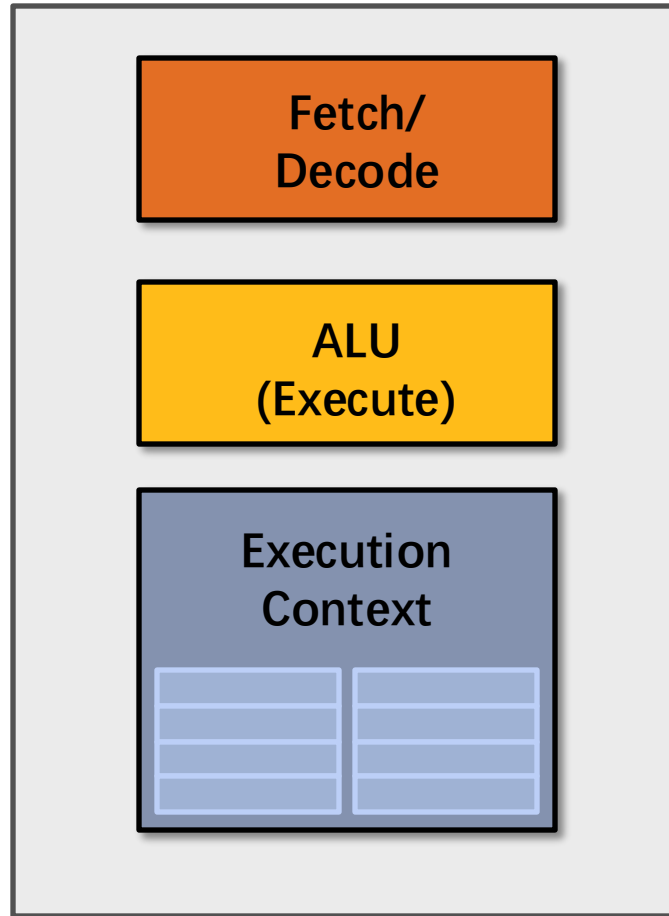
Instruction stream sharing



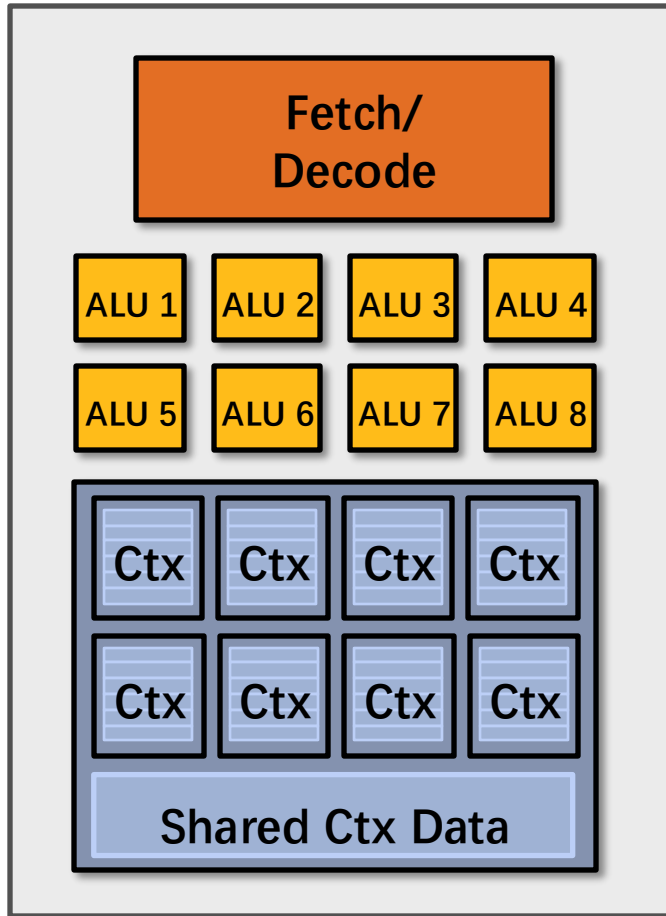
But ... many fragments should be able to share an instruction stream!

```
<diffuseShader>:  
sample r0, v4, t0, s0  
mul  r3, v0, cb0[0]  
madd r3, v1, cb0[1], r3  
madd r3, v2, cb0[2], r3  
clmp r3, r3, 1(0.0), 1(1.0)  
mul  o0, r0, r3  
mul  o1, r1, r3  
mul  o2, r2, r3  
mov  o3, 1(1.0)
```

Recall: simple processing core



Add ALUs

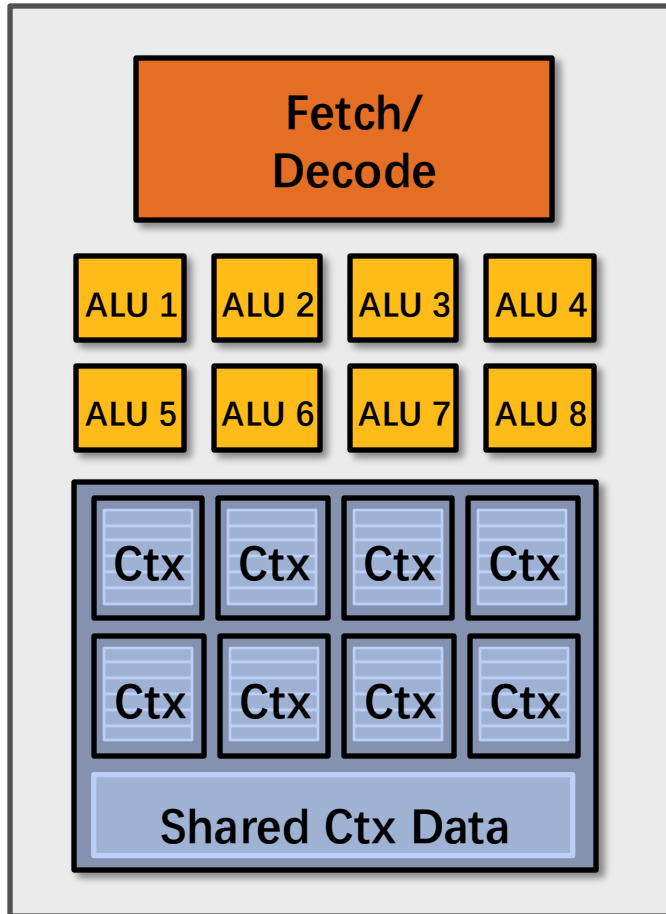


Idea #2:

Amortize cost/complexity of managing an instruction stream across many ALUs

SIMD processing

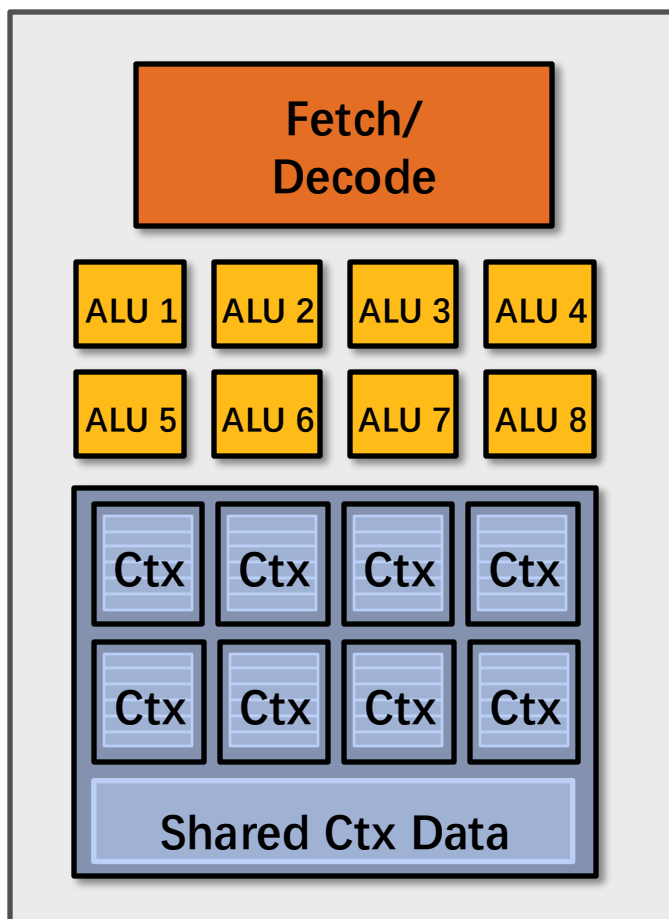
Modifying the shader



```
<diffuseShader>:  
sample r0, v4, t0, s0  
mul  r3, v0, cb0[0]  
madd r3, v1, cb0[1], r3  
madd r3, v2, cb0[2], r3  
clmp r3, r3, l(0.0), l(1.0)  
mul  o0, r0, r3  
mul  o1, r1, r3  
mul  o2, r2, r3  
mov  o3, l(1.0)
```

Original compiled shader:
Processes one fragment using
scalar ops on scalar registers

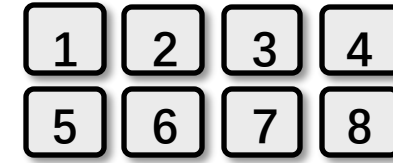
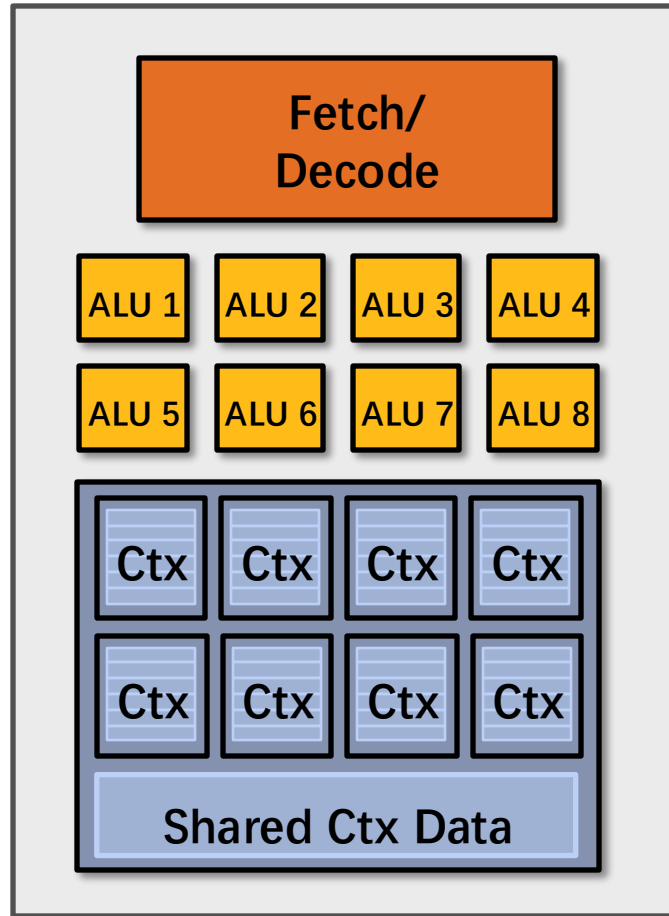
Modifying the shader



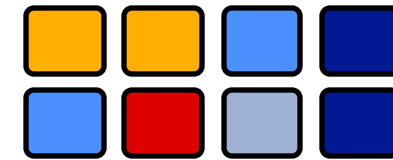
```
<VEC8_diffuseShader>:  
VEC8_sample vec_r0, vec_v4, t0, vec_s0  
VEC8_mul vec_r3, vec_v0, cb0[0]  
VEC8_madd vec_r3, vec_v1, cb0[1], vec_r3  
VEC8_madd vec_r3, vec_v2, cb0[2], vec_r3  
VEC8_clamp vec_r3, vec_r3, 1(0.0), 1(1.0)  
VEC8_mul vec_o0, vec_r0, vec_r3  
VEC8_mul vec_o1, vec_r1, vec_r3  
VEC8_mul vec_o2, vec_r2, vec_r3  
VEC8_mov o3, 1(1.0)
```

New compiled shader:
Processes eight fragments using
vector ops on vector registers

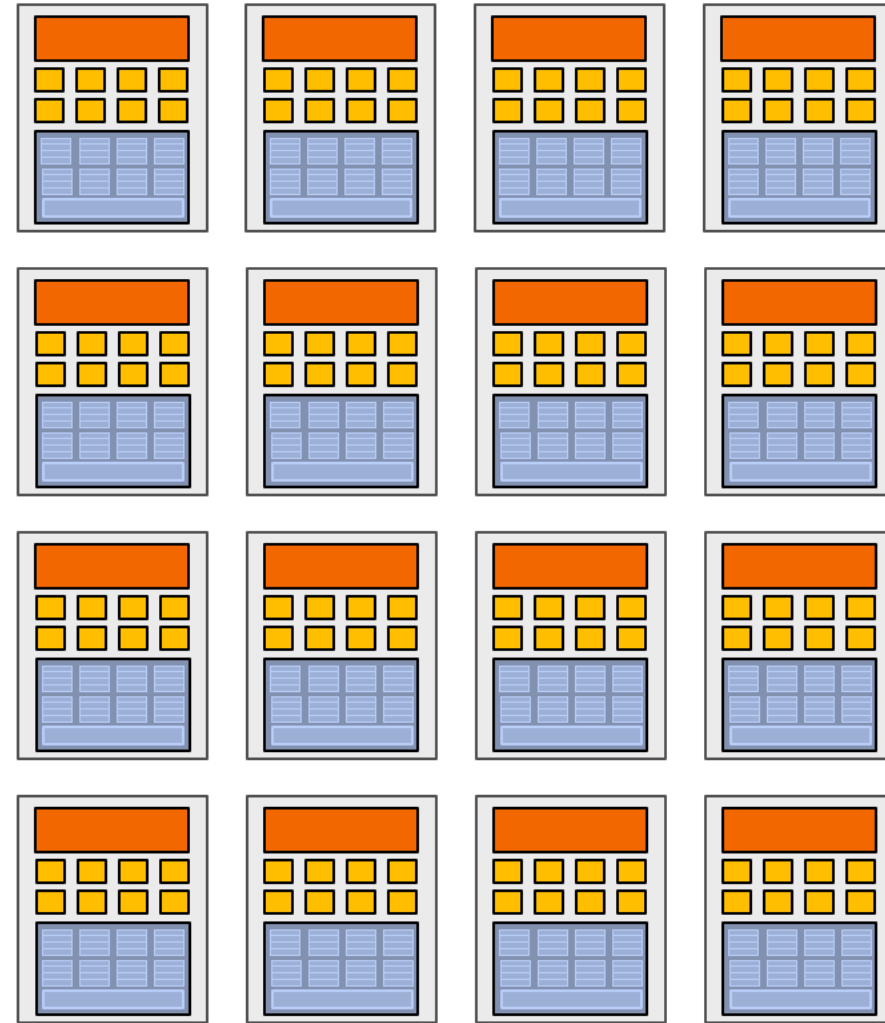
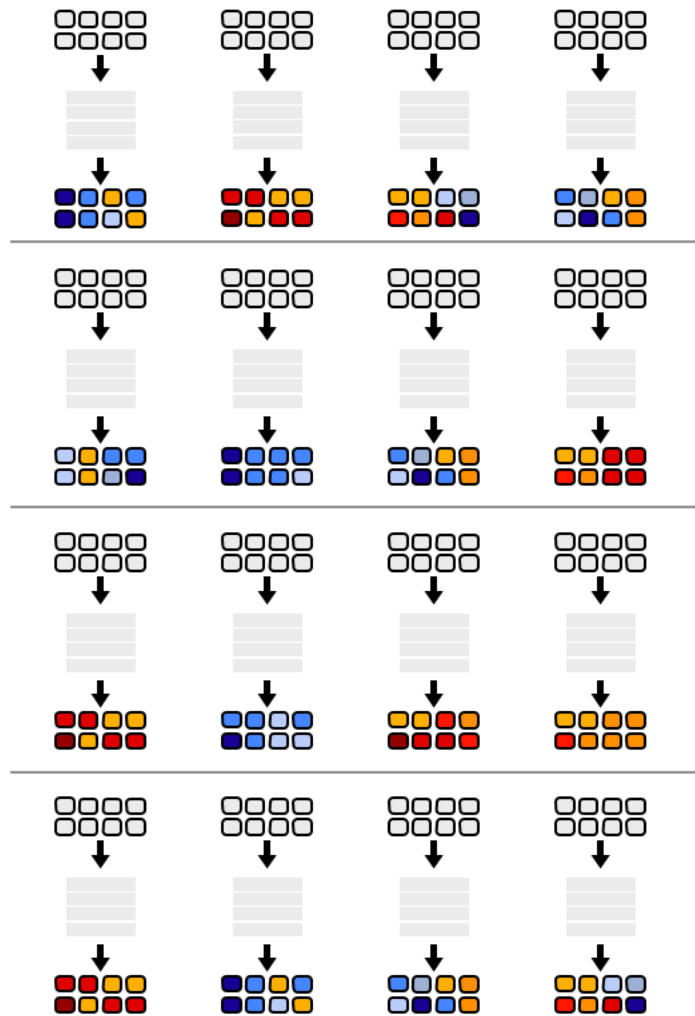
Modifying the shader



```
<VEC8_diffuseShader>:  
VEC8_sample vec_r0, vec_v4, t0, vec_s0  
VEC8_mul  vec_r3, vec_v0, cb0[0]  
VEC8_madd vec_r3, vec_v1, cb0[1], vec_r3  
VEC8_madd vec_r3, vec_v2, cb0[2], vec_r3  
VEC8_clmp vec_r3, vec_r3, l(0.0), l(1.0)  
VEC8_mul  vec_o0, vec_r0, vec_r3  
VEC8_mul  vec_o1, vec_r1, vec_r3  
VEC8_mul  vec_o2, vec_r2, vec_r3  
VEC8_mov  o3, l(1.0)
```



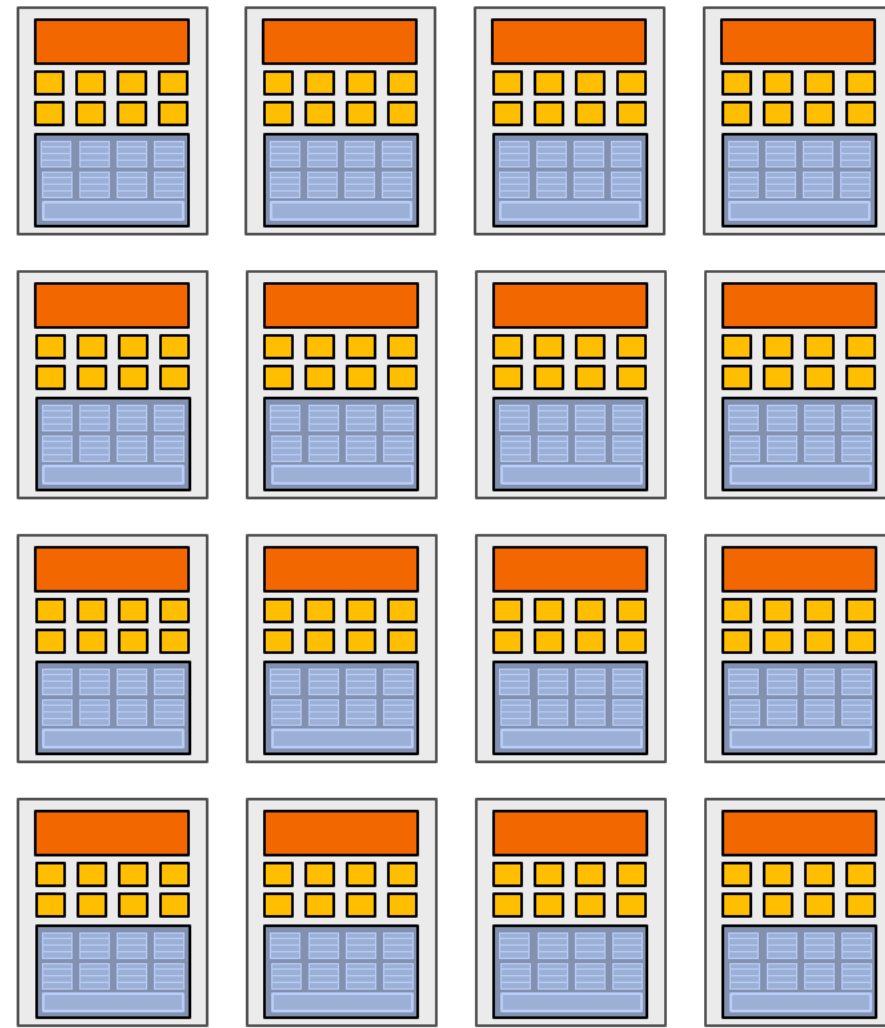
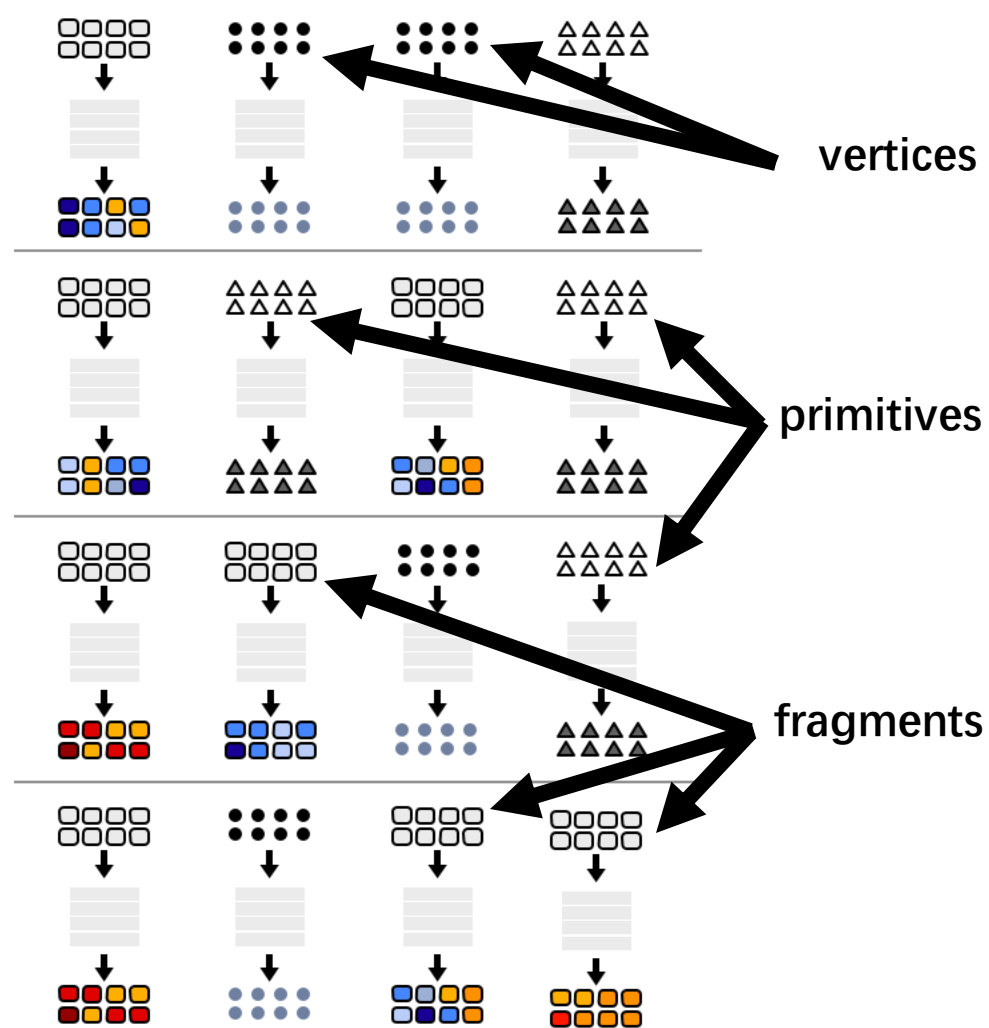
128 fragments in parallel



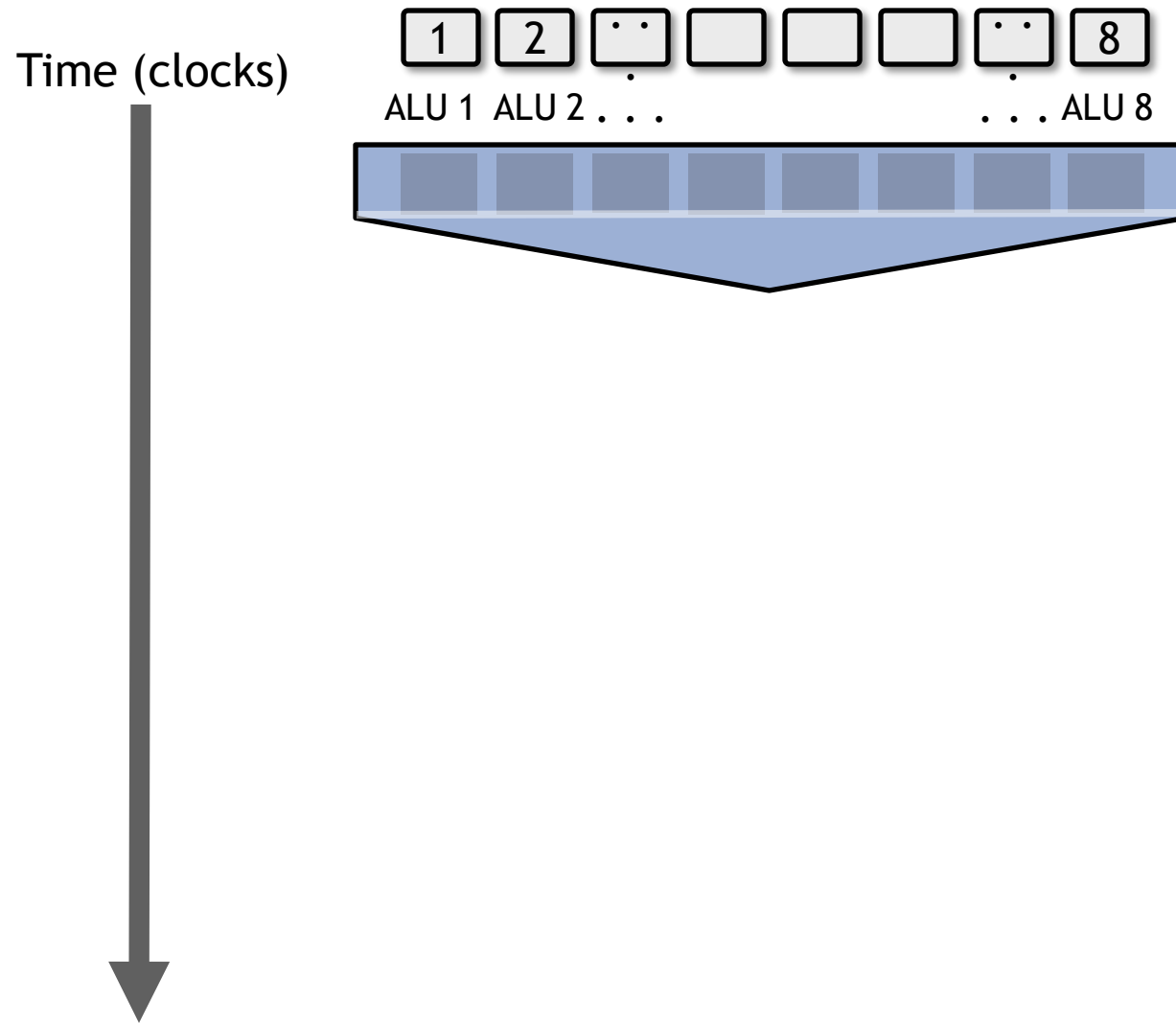
16 cores = 128 ALUs, 16 simultaneous instruction streams

vertices/fragments
primitives
OpenCL work items
CUDA threads

128 fragments in parallel



But what about branches?

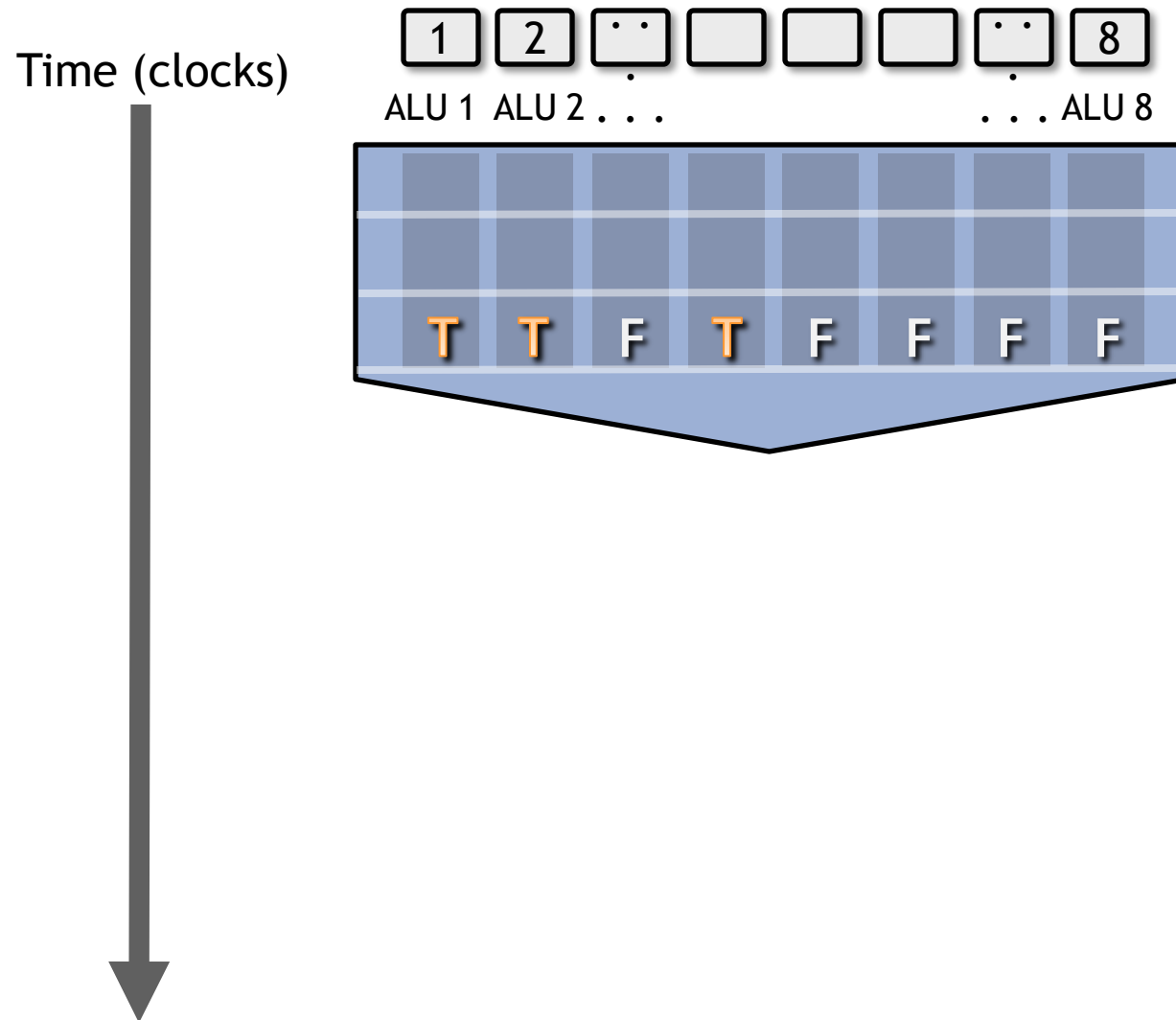


<unconditional
shader code>

```
if (x > 0) {  
    y = pow(x, exp);  
    y *= Ks;  
    refl = y + Ka;  
} else {  
    x = 0;  
    refl = Ka;  
}
```

<resume unconditional
shader code>

But what about branches?

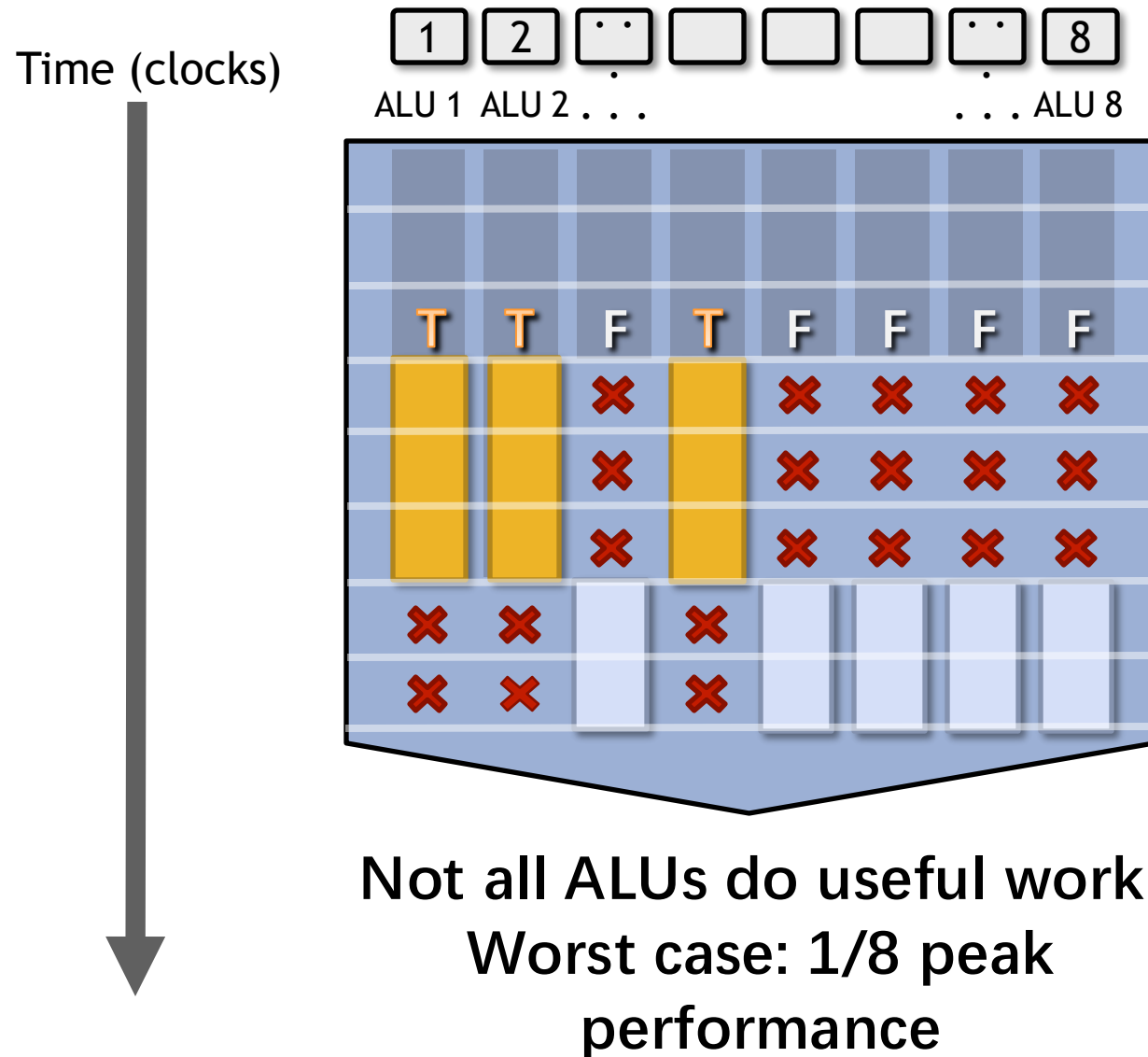


<unconditional
shader code>

```
if (x > 0) {  
    y = pow(x, exp);  
    y *= Ks;  
    refl = y + Ka;  
} else {  
    x = 0;  
    refl = Ka;  
}
```

<resume unconditional
shader code>

But what about branches?

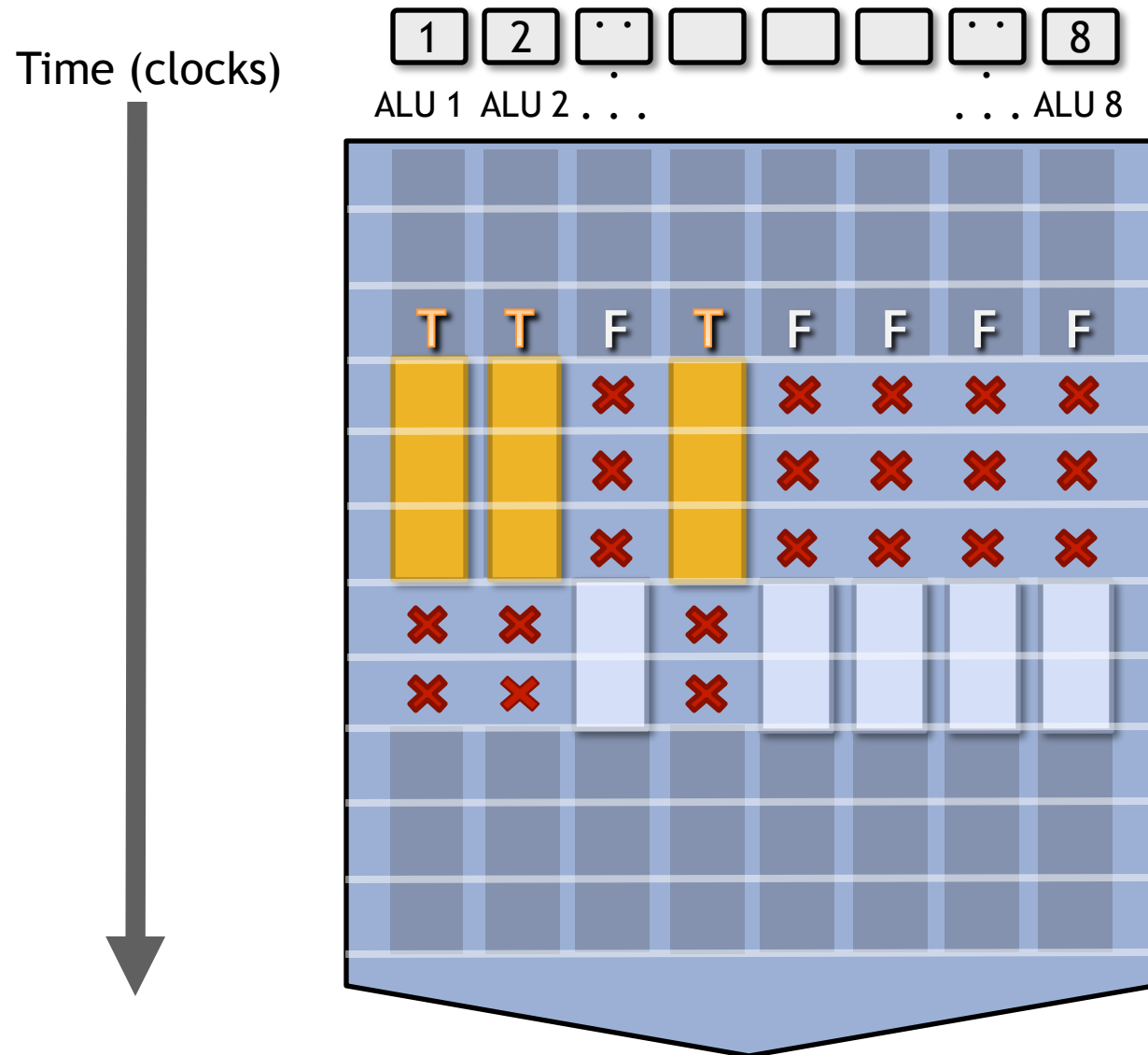


<unconditional
shader code>

```
if (x > 0) {  
    y = pow(x, exp);  
    y *= Ks;  
    refl = y + Ka;  
} else {  
    x = 0;  
    refl = Ka;  
}
```

<resume unconditional
shader code>

But what about branches?



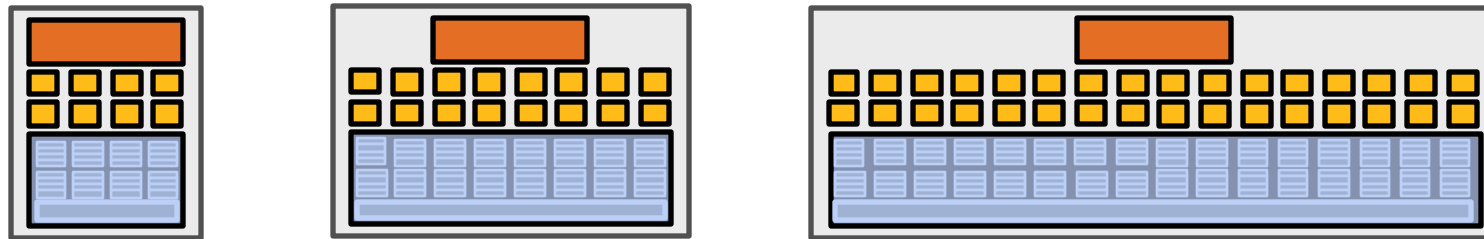
<unconditional
shader code>

```
if (x > 0) {  
    y = pow(x, exp);  
    y *= Ks;  
    refl = y + Ka;  
} else {  
    x = 0;  
    refl = Ka;  
}
```

<resume unconditional
shader code>

Clarification

- SIMD processing does not imply SIMD instructions
- Option 1: explicit vector instructions
 - ▣ x86 AVX, ARM NEON, RISC V Vectors
- Option 2: scalar instructions, implicit HW vectorization
 - ▣ HW determines instruction stream sharing across ALUs (amount of sharing hidden from software)
 - ▣ NVIDIA GeForce (“SIMT” warps), ATI Radeon architectures (“wavefronts”)



In practice: 16 to 64 fragments share an instruction stream.

What about *stalls*?

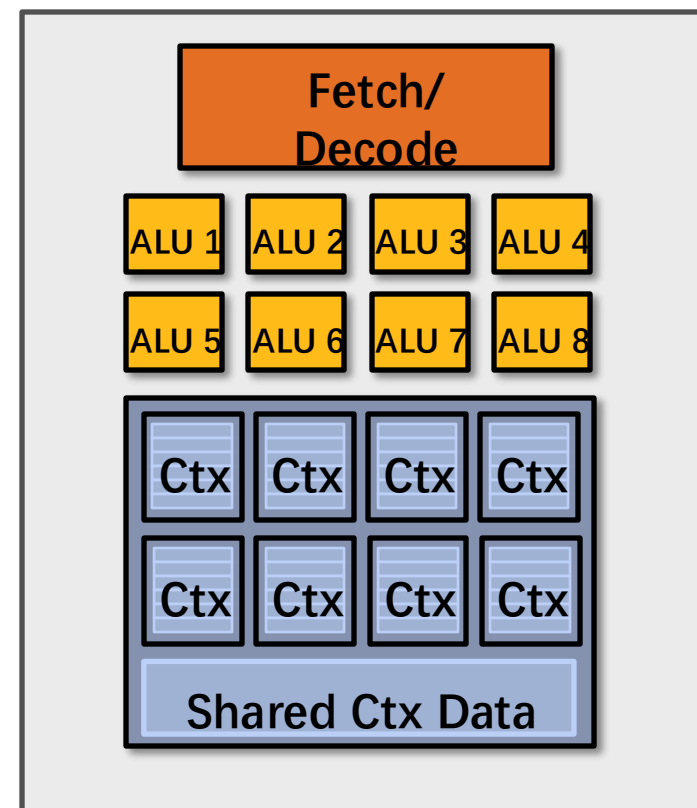
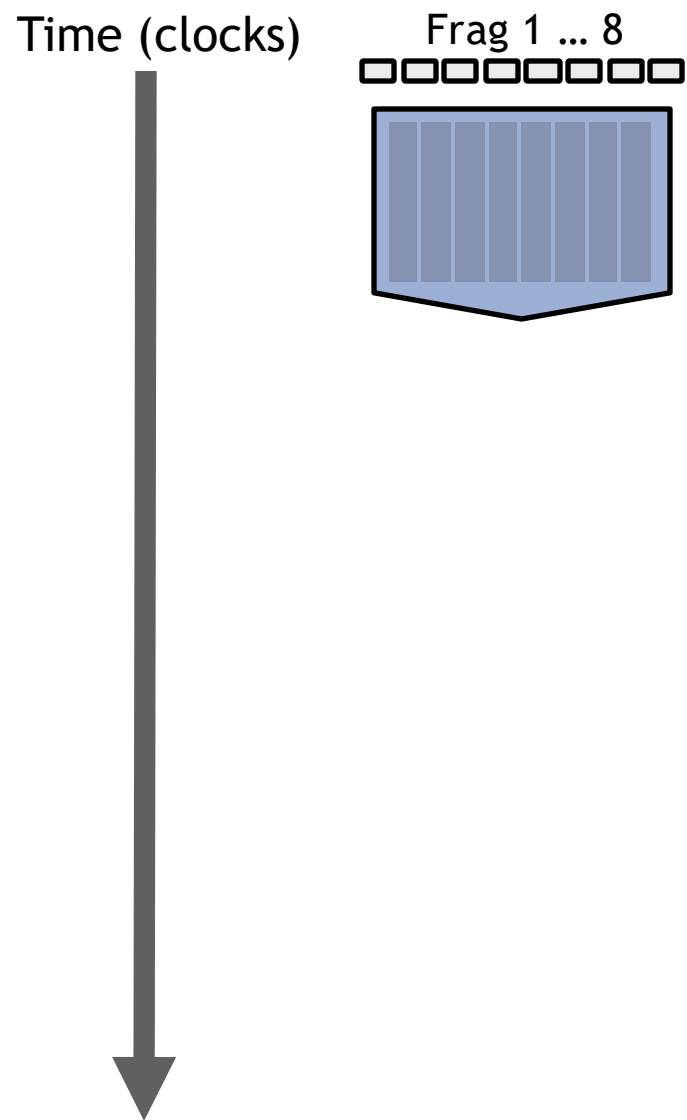
- Stalls occur when a core cannot run the next instruction because of a dependency on a previous operation.
- Texture access latency = 100s of cycles
- We've removed the fancy caches and logic that helps avoid stalls

But we have LOTS of independent fragments!

Idea #3:

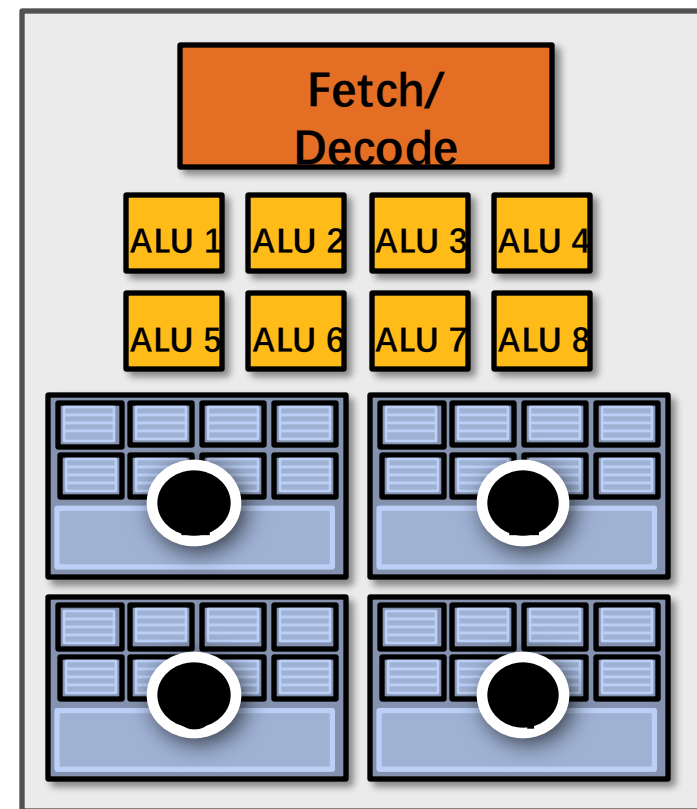
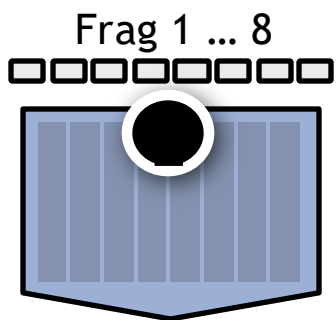
Interleave processing of many fragments on a single core to avoid stalls caused by high latency operations.

Hiding shader stalls

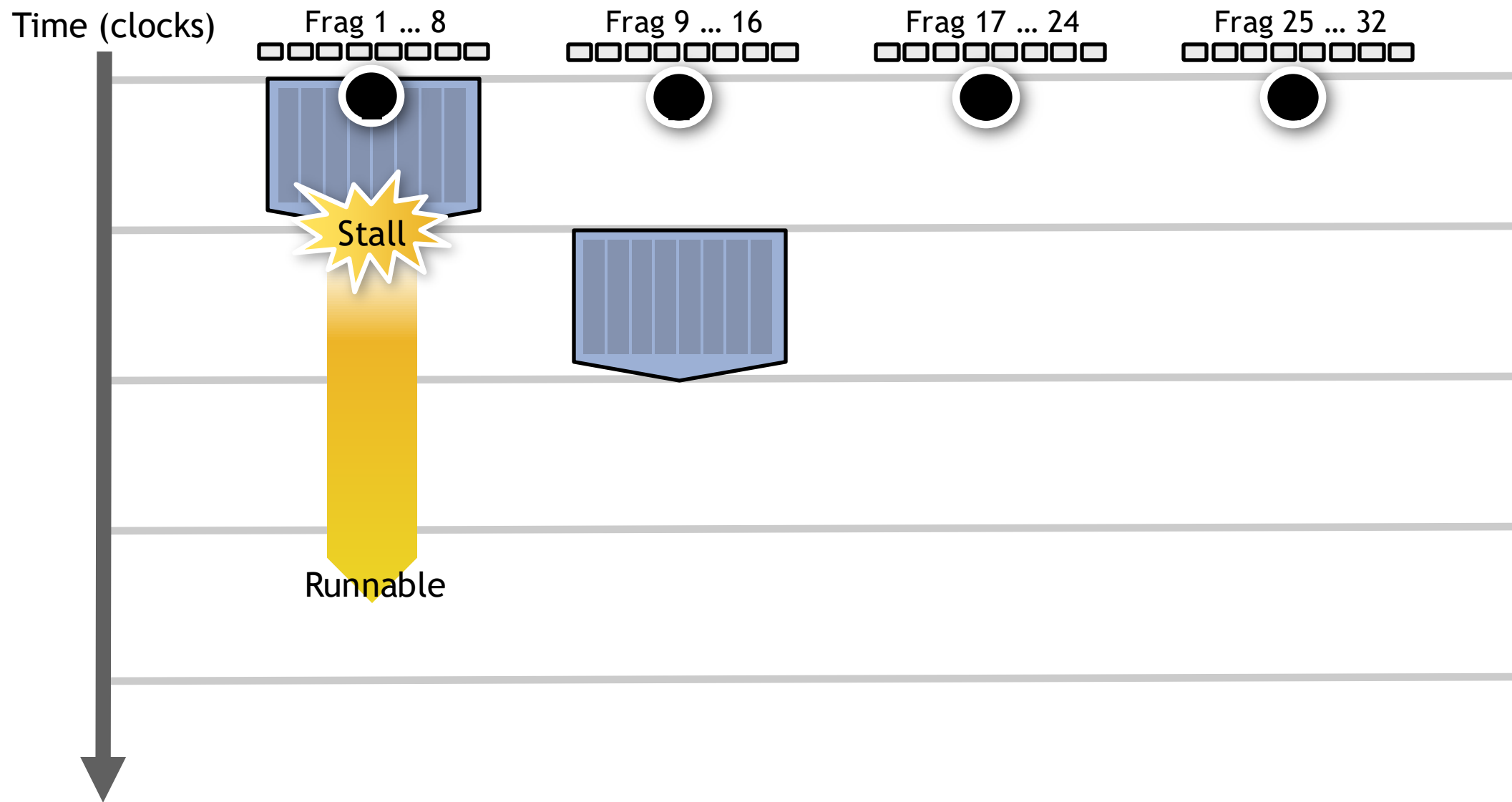


Hiding shader stalls

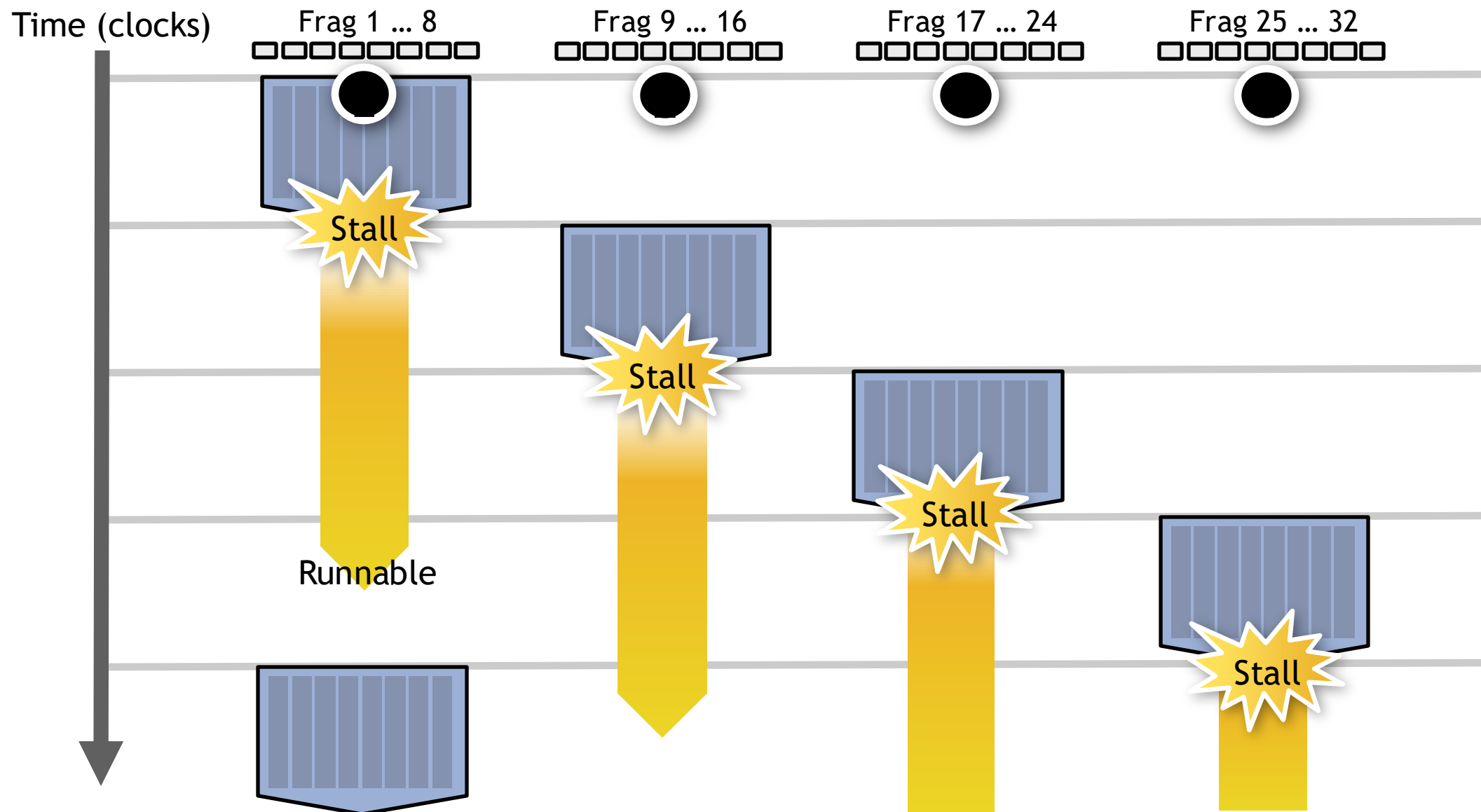
Time (clocks)



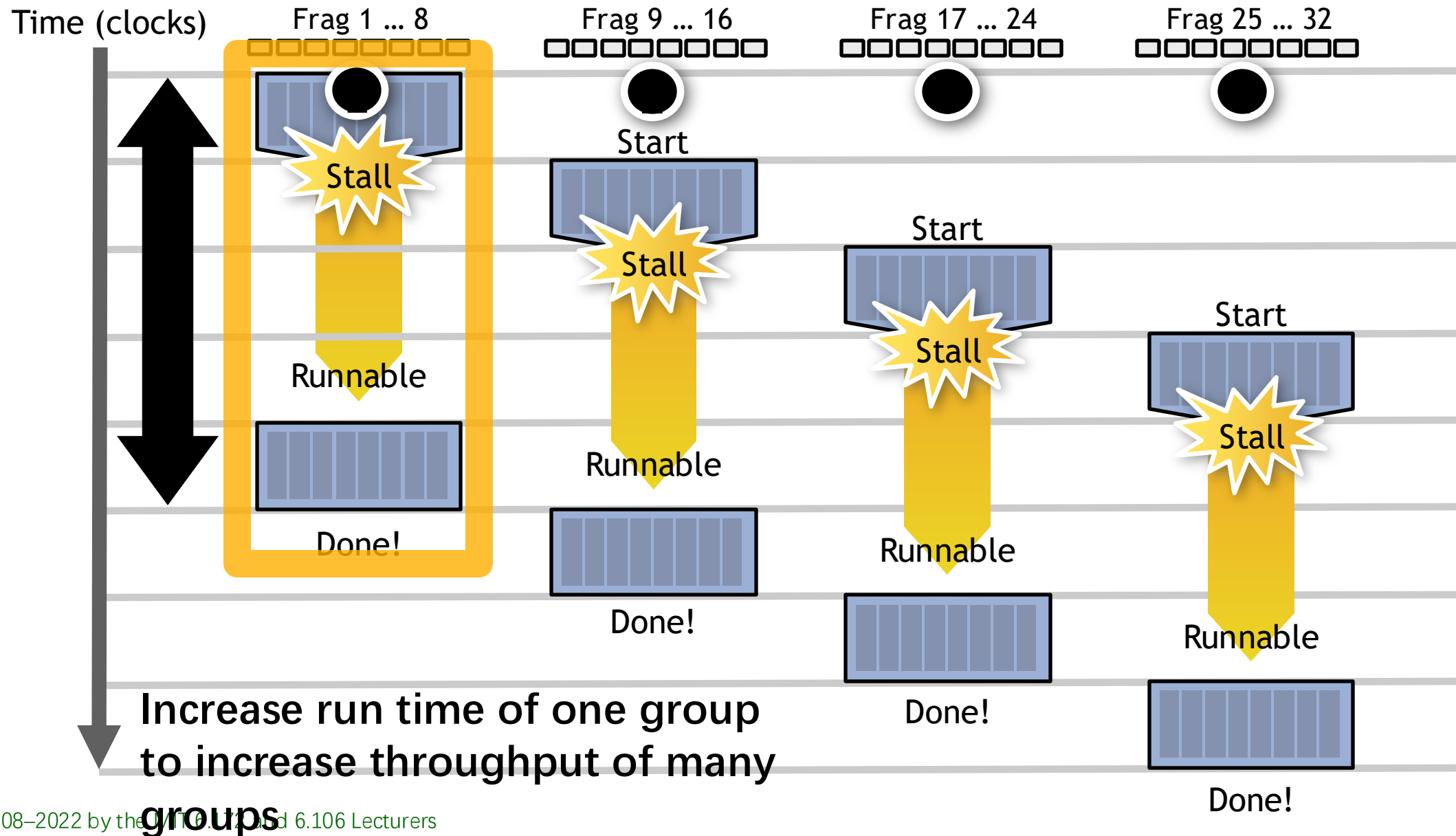
Hiding shader stalls



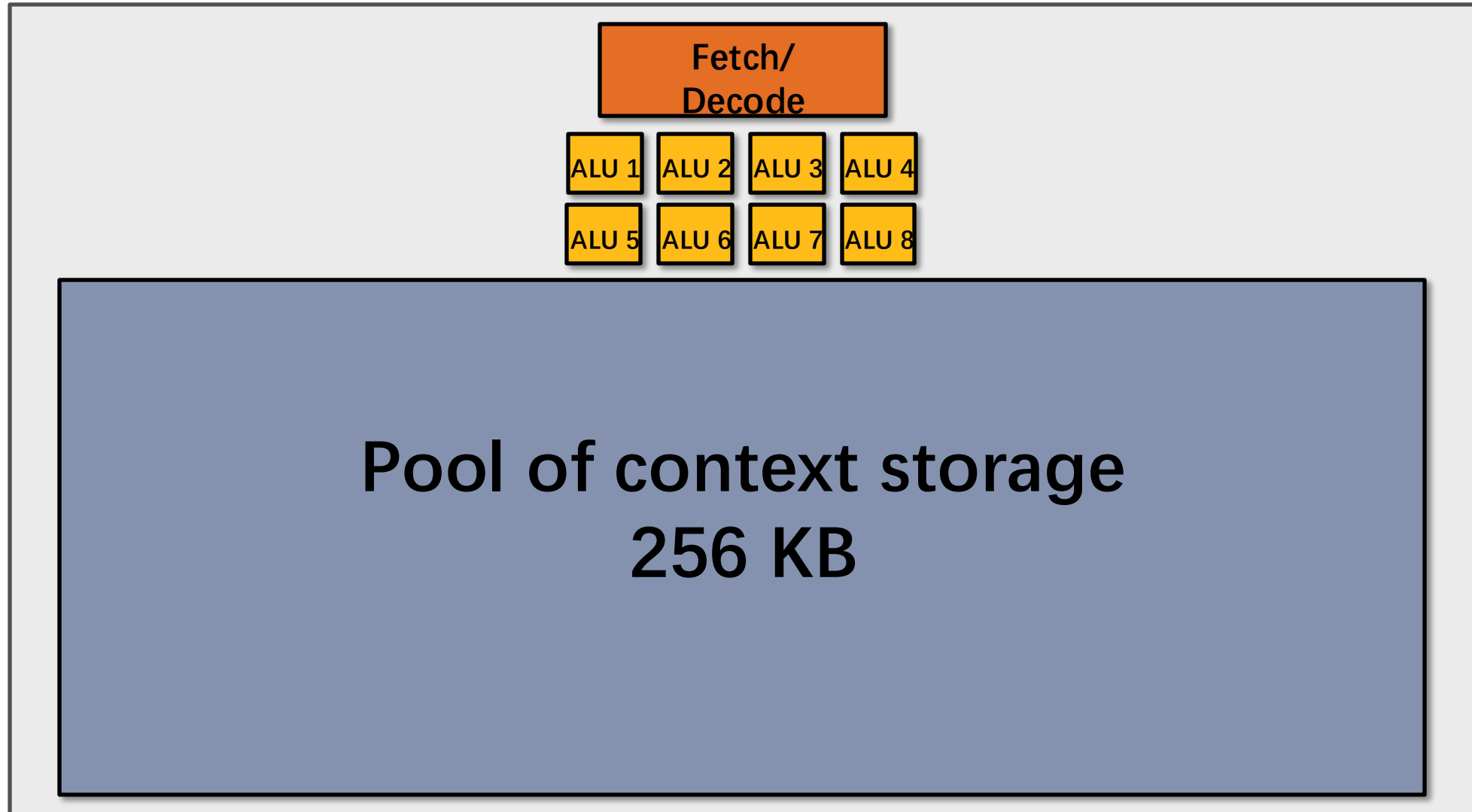
Hiding shader stalls



Throughput!

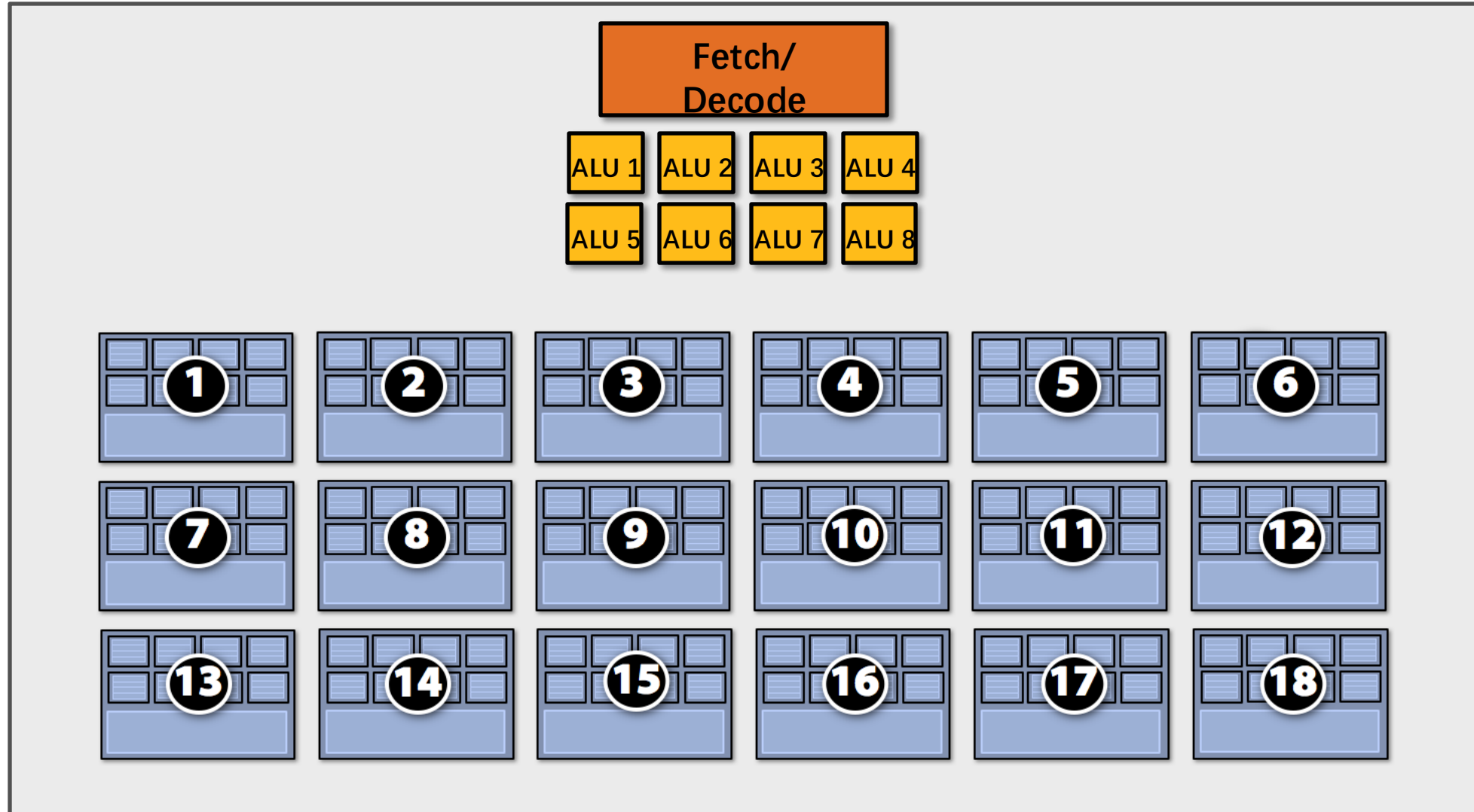


Storing contexts

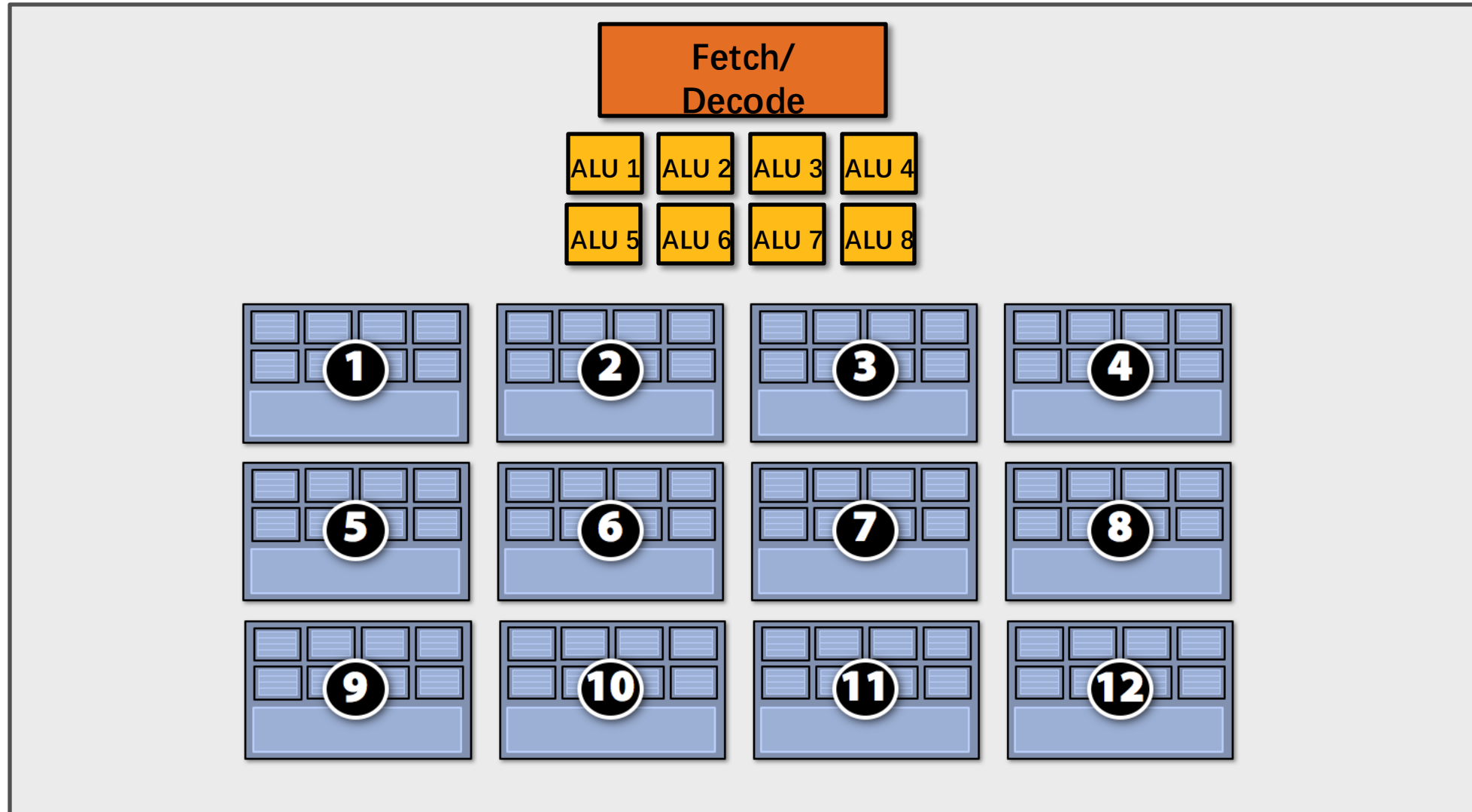


Eighteen small contexts

(maximal latency hiding)

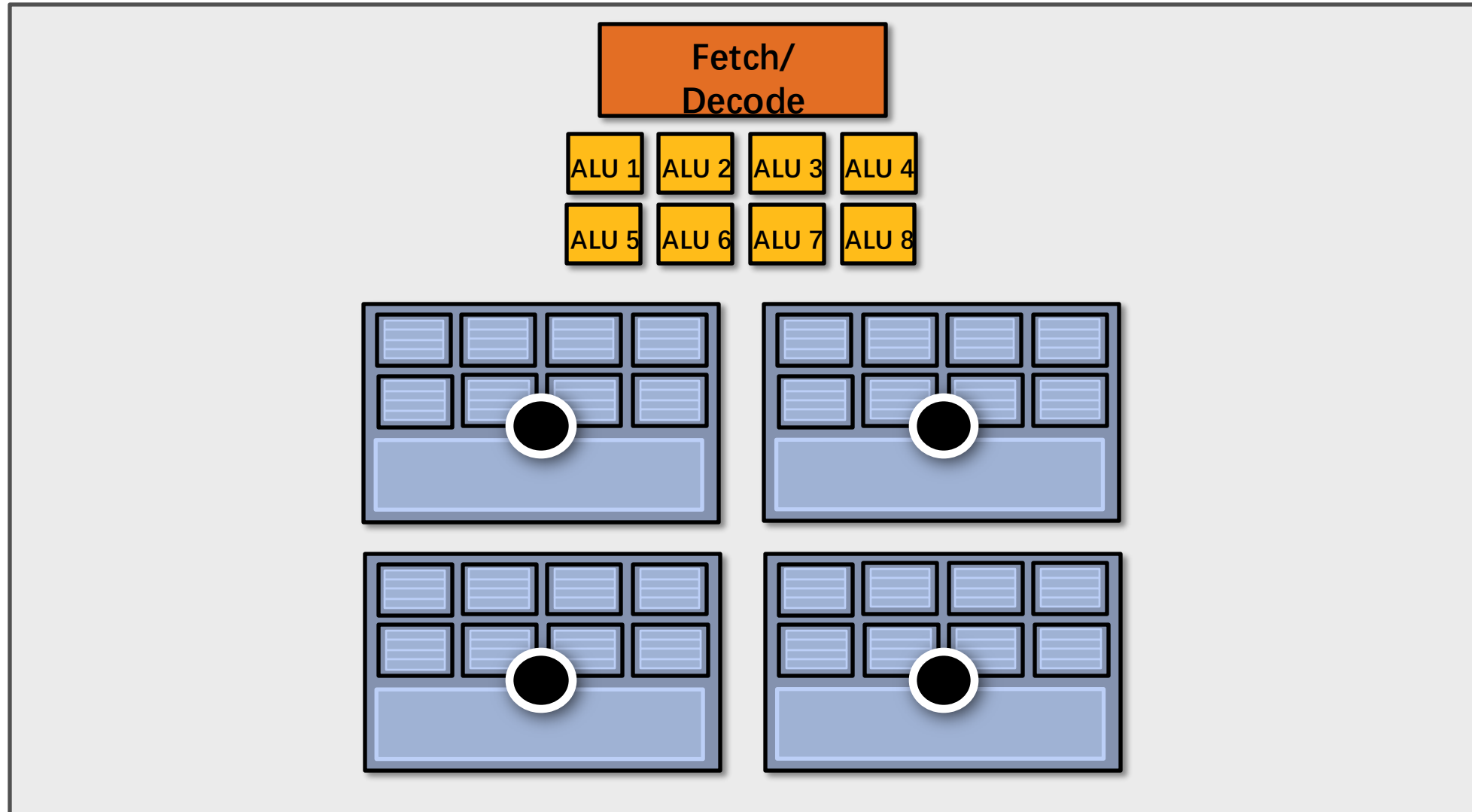


Twelve medium contexts



Four large contexts

(low latency hiding ability)



Clarification

- Interleaving between contexts can be managed by hardware or software (or both!)
- NVIDIA / ATI Radeon GPUs
 - ▣ HW schedules / manages all contexts (lots of them)
 - ▣ Special on-chip storage holds fragment state
- Intel Larrabee
 - ▣ HW manages four x86 (big) contexts at fine granularity
 - ▣ SW scheduling interleaves many groups of fragments on each HW context
 - ▣ L1-L2 cache holds fragment state (as determined by SW)

My GPU!

16 cores

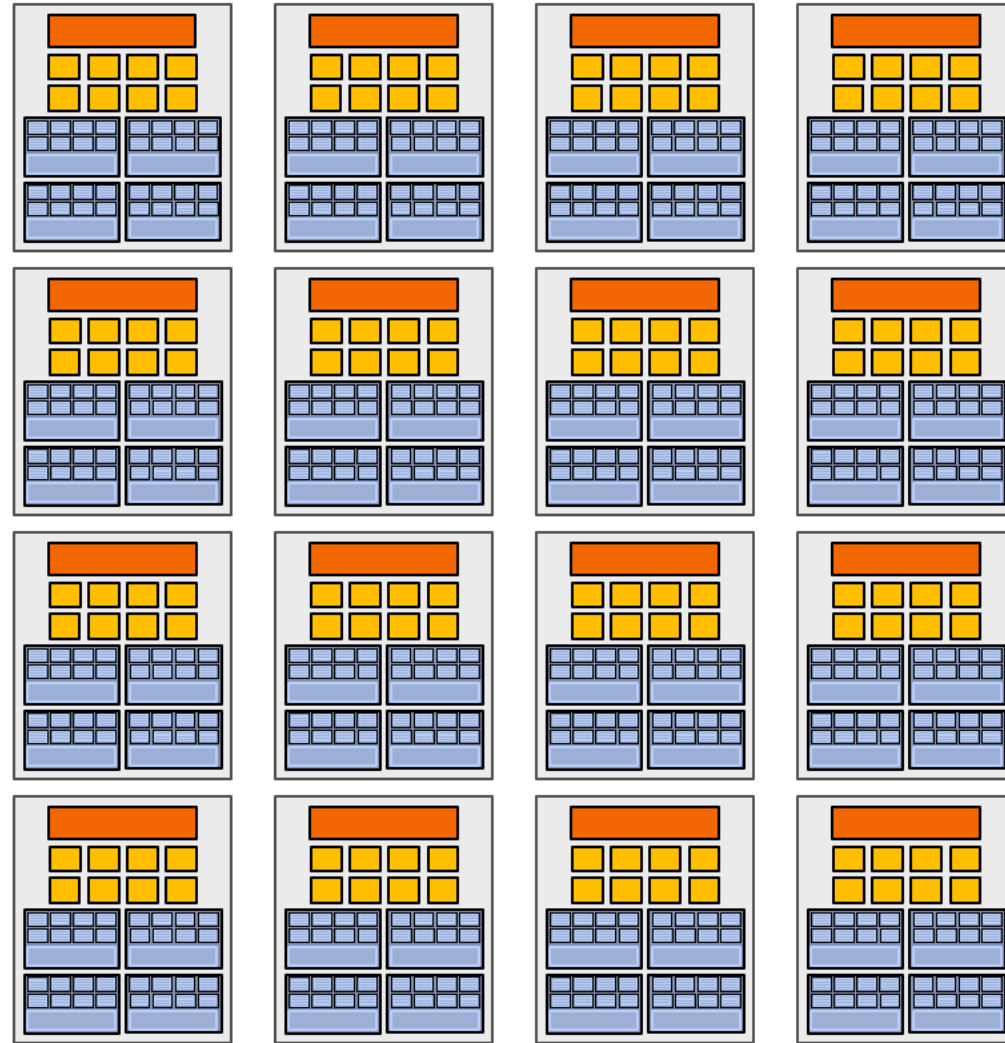
8 mul-add ALUs per core
(128 total)

16 simultaneous
instruction streams

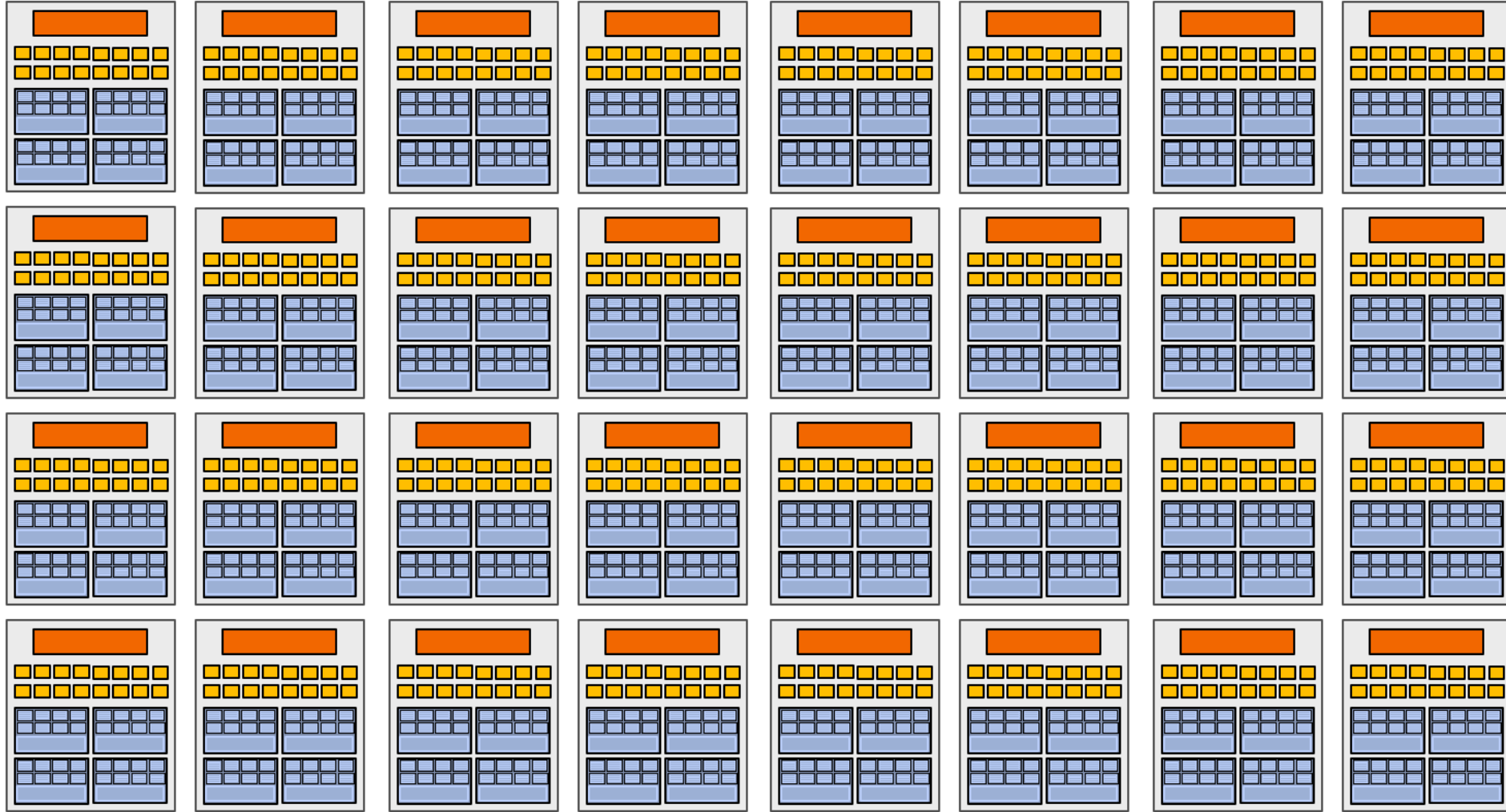
64 concurrent (but interleaved)
instruction streams

512 concurrent fragments

= 256 GFLOPs (@ 1GHz)



My bigger GPU!



32 cores, 16 ALUs per core (512 total) = 1 TFLOP (@ 1 GHz)

Summary: three key ideas

- Use many “slimmed down cores” to run in parallel
- Pack cores full of ALUs (by sharing instruction stream across groups of fragments)
 - ▣ Option 1: Explicit SIMD vector instructions
 - ▣ Option 2: Implicit sharing managed by hardware
- Avoid latency stalls by interleaving execution of many groups of fragments
 - ▣ When one group stalls, work on another group

PUTTING THREE IDEAS INTO PRACTICE: A CLOSER LOOK AT A REAL GPU

NVIDIA GEFORCE GTX 1080

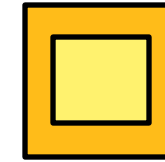
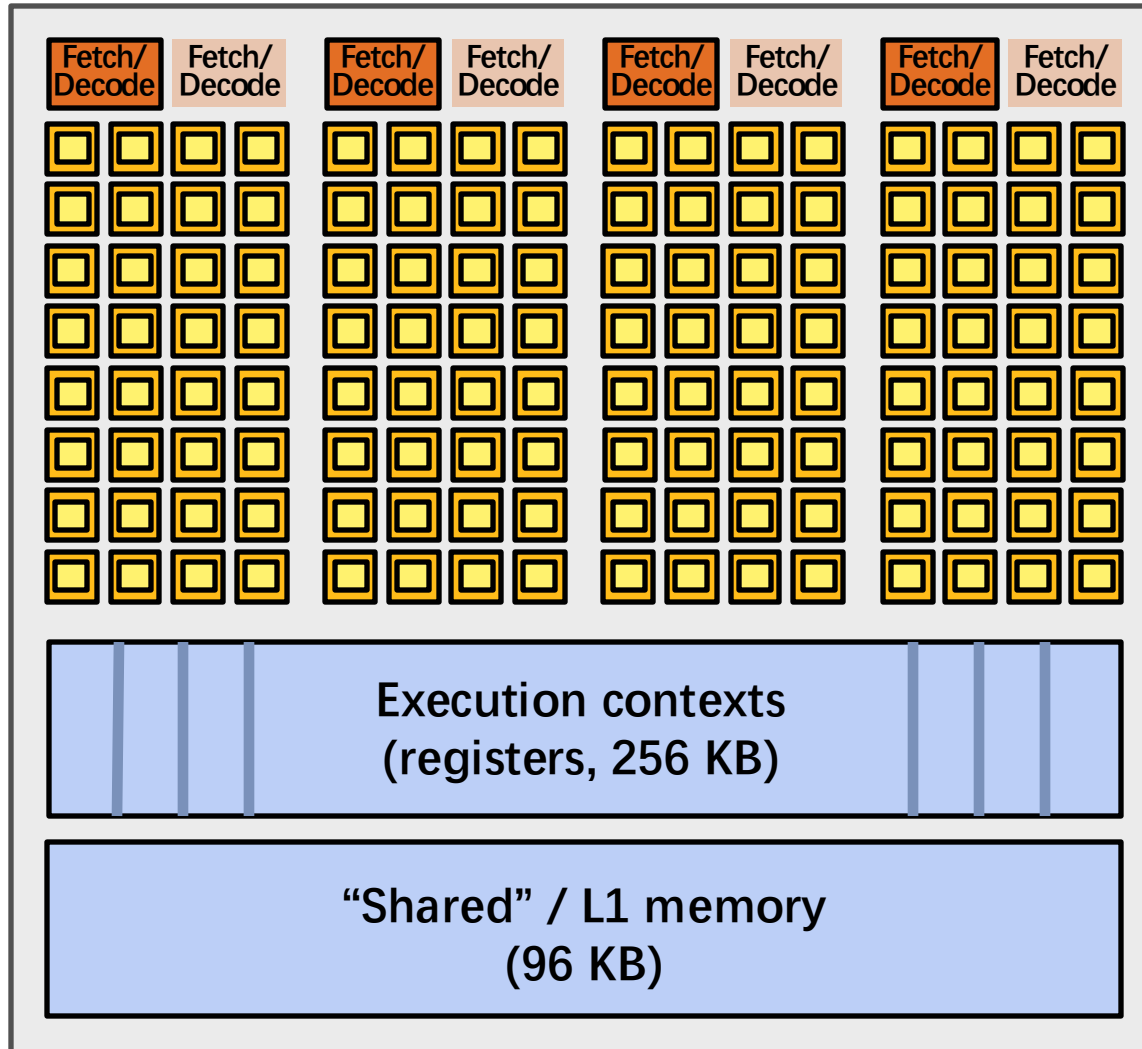


NVIDIA GeForce GTX 1080

- NVIDIA-speak:
 - ▣ 2560 stream processors (“CUDA cores”)
 - ▣ “SIMT execution”
- Generic speak:
 - ▣ 20 cores
 - ▣ 4 groups of 32 SIMD functional units per core



NVIDIA GeForce GTX 1080 “core”



= SIMD function unit,
control shared across 32 units
(1 MUL-ADD per clock)

- Groups of 32 [fragments/vertices/CUDA threads] share an instruction stream
- Up to 64 groups are simultaneously interleaved
- Up to 2,048 individual contexts can be stored

NVIDIA GeForce GTX 1080



There are 20 of these things on the GTX 1080

That's 40,960 fragments!

(Or 40,960 “CUDA threads”)

NVIDIA GeForce RTX 3080 “core”



NVIDIA GeForce RTX 3080



There are *68* of these things on the RTX 3080

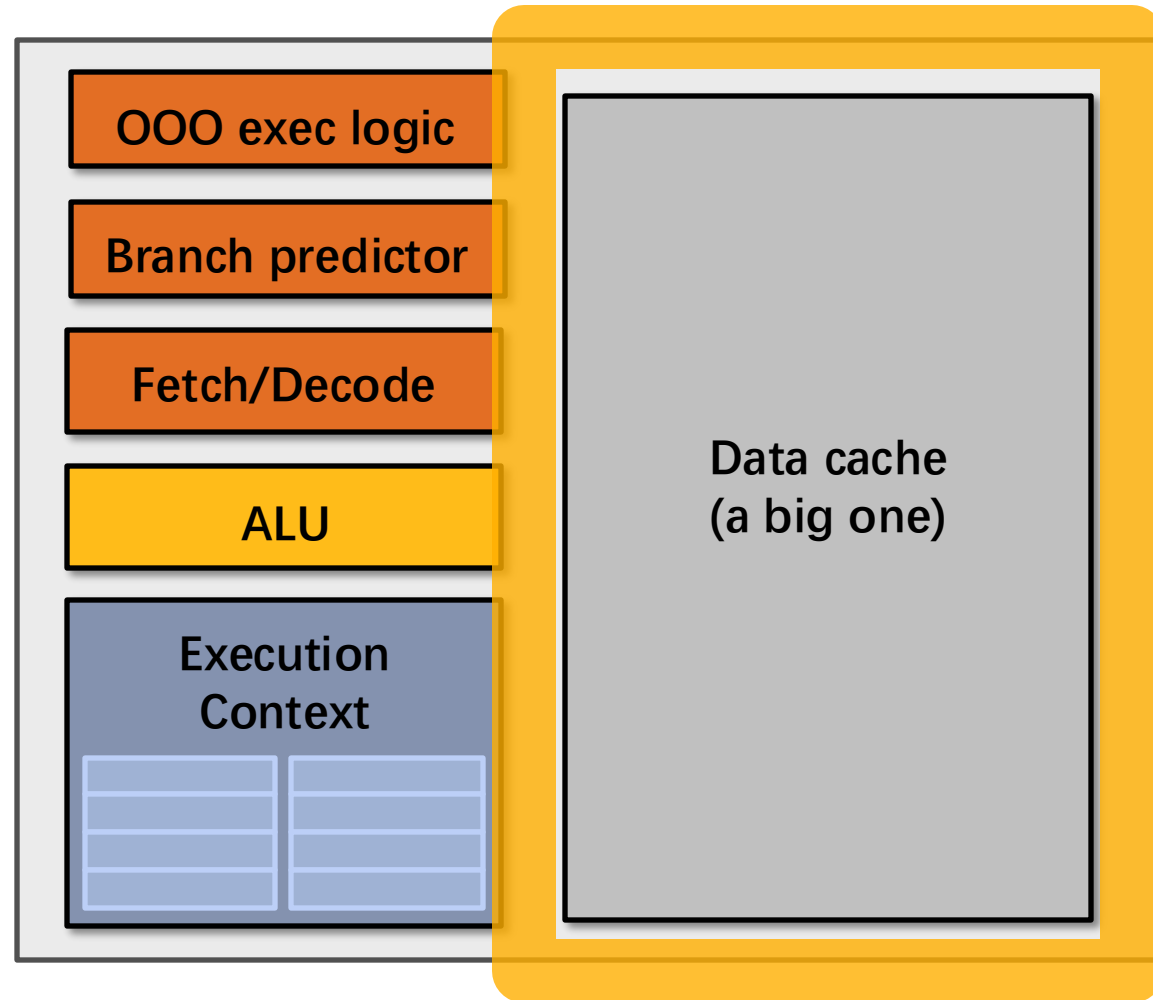
That's 139,264 fragments!

(Or 139,264 “CUDA threads”)

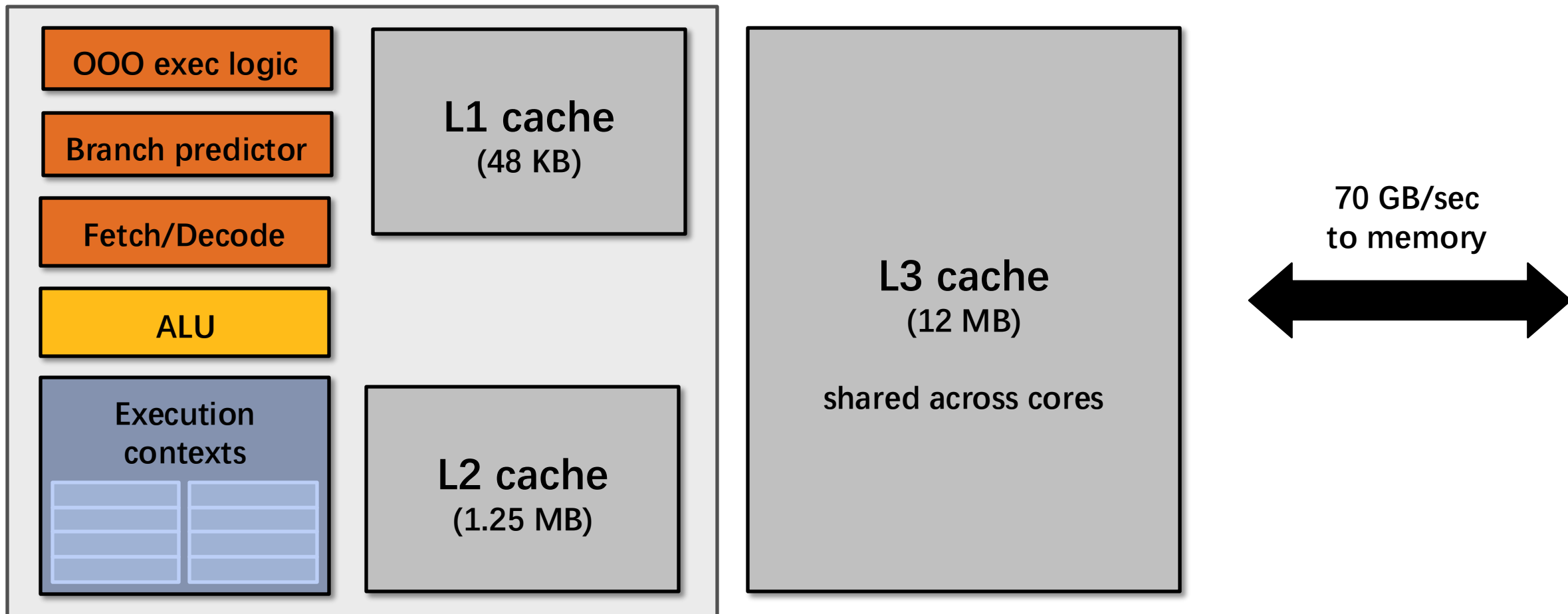
MOVING DATA TO PROCESSORS



Recall: “CPU-style” core

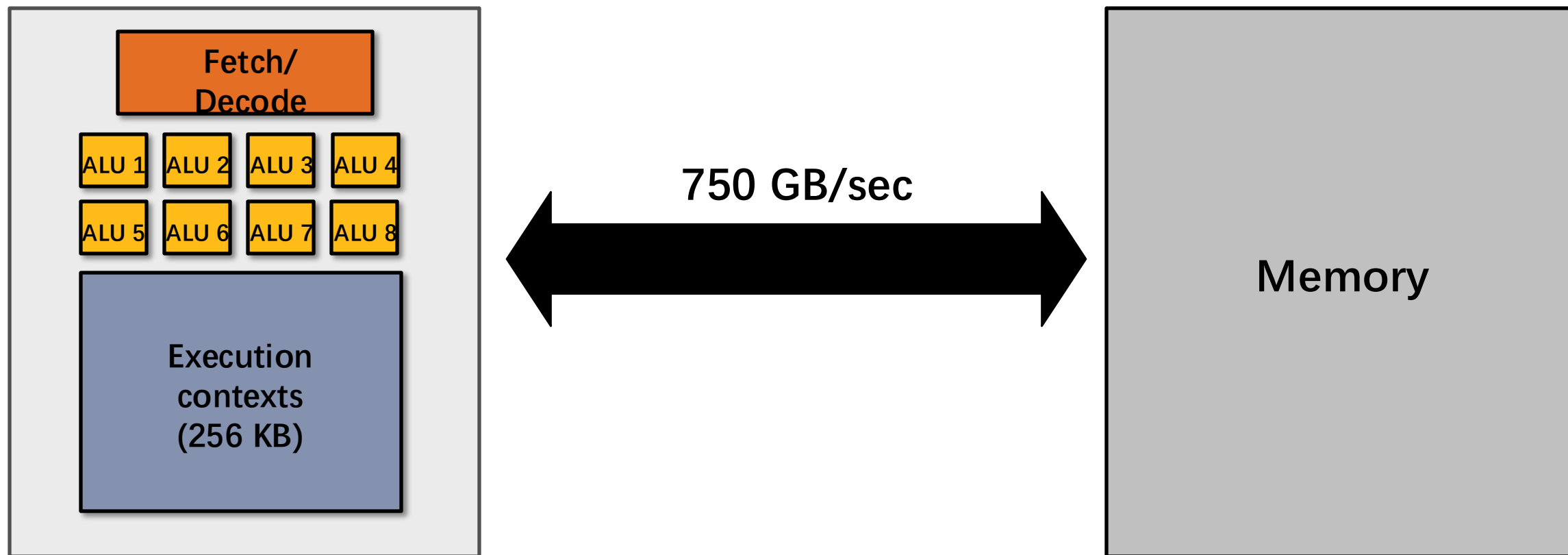


“CPU-style” memory hierarchy



CPU cores run efficiently when data is resident in cache
(caches reduce latency, provide high bandwidth)

Throughput core (GPU-style)



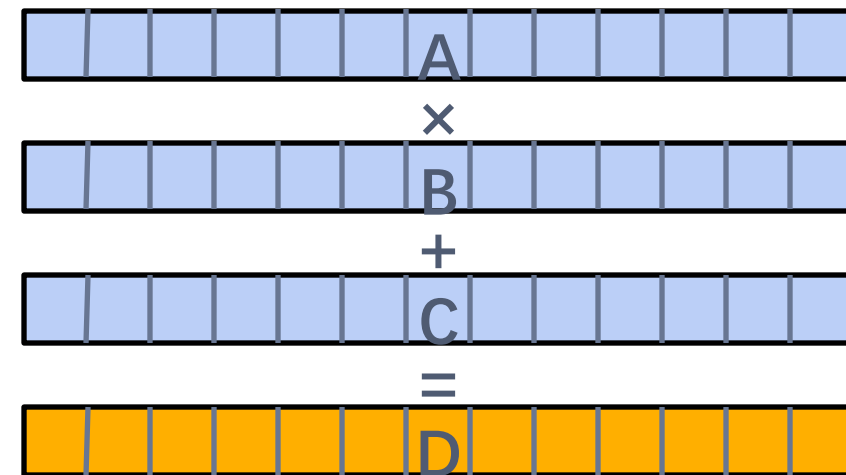
More ALUs, no large traditional cache hierarchy:
Need high-bandwidth connection to memory

Bandwidth is a critical resource

- A high-end GPU (e.g. RTX 3080) has...
 - About sixty times (30 TFLOPS) the compute performance of 4-core CPU
 - No large cache hierarchy to absorb memory requests
- GPU memory system is designed for throughput
 - Wide bus (750 GB/sec)
 - Repack/reorder/interleave memory requests to maximize use of mem. bus
 - Still, this is only ten times the bandwidth available to CPU

Bandwidth thought experiment

- Task: element-wise multiply two long vectors A and B
 - ▣ Load input A[i]
 - ▣ Load input B[i]
 - ▣ Load input C[i]
 - ▣ Compute $A[i] \times B[i] + C[i]$
 - ▣ Store result into D[i]
- Four memory operations (16 bytes) for every MUL-ADD
- RTX 3080 can do 8704 MUL-ADDS per clock
- Need ~240 TB/sec of bandwidth to keep functional units busy
- Roughly 0.33% efficiency... but 10x faster than CPU!



Bandwidth limited!

If processors request data at too high a rate,
the memory system cannot keep up.

No amount of latency hiding helps this.

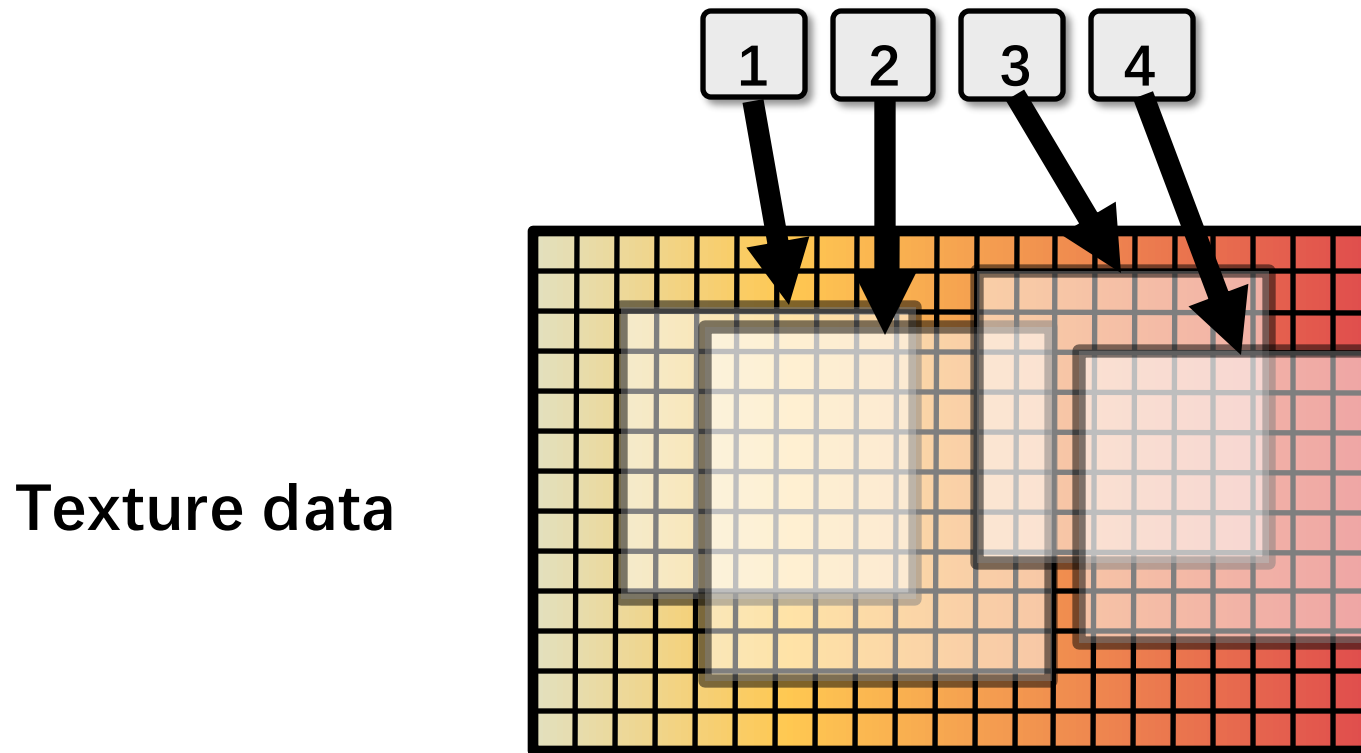
Overcoming bandwidth limits are a common challenge
for GPU-compute application developers.

Reducing bandwidth requirements

- Request data less often (instead, do more math)
 - ▣ “arithmetic intensity”
- Fetch data from memory less often (share/reuse data across fragments)
 - ▣ on-chip communication or storage

Reducing bandwidth requirements

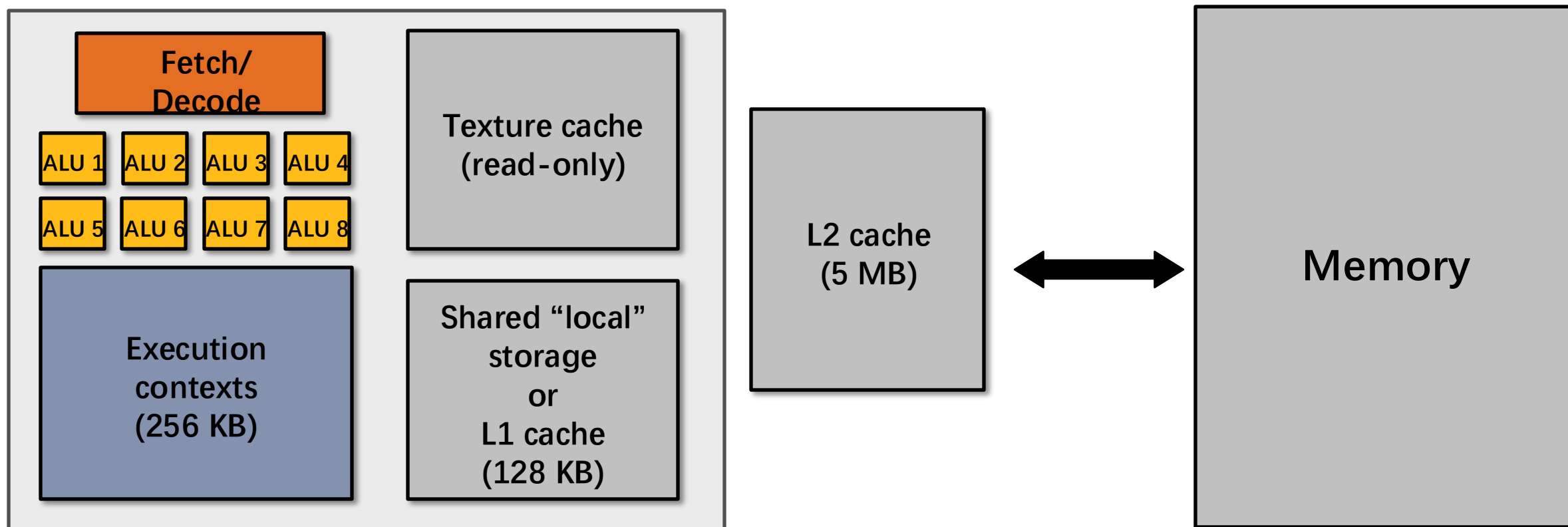
- Two examples of on-chip storage
 - ▣ Texture caches
 - ▣ Scratchpad (OpenCL “local memory,” CUDA “shared memory”)



Texture caches:

Capture reuse across fragments, not temporal reuse within a single shader program

Modern GPU memory hierarchy



- On-chip storage takes load off memory system
- Many developers calling for more cache-like storage
- (particularly GPU-compute applications)

Summary: GPU Architecture

- Think of a GPU as a heterogeneous multi-core processor optimized for maximum throughput.
 - ▣ (currently at the extreme end of the design space)
- Think about how the various programming models map to these architectural characteristics

Summary: GPU Architecture

- An efficient GPU workload ...
 - ▣ Has thousands of independent pieces of work
 - ◆ Uses many ALUs on many cores
 - ◆ Supports massive interleaving for latency hiding
 - ▣ Is amenable to instruction stream sharing
 - ◆ Maps to SIMD execution well
 - ▣ Is compute-heavy: the ratio of math operations to memory access is high
 - ◆ Not limited by memory bandwidth